Unit 1 **Introduction to Logic Design Using Verilog** 1.HDL 2.Logic Elements 3.Comments 4.Logic Gates 5.Logic Macro Functions 6.Procedural Flow Control 7.Net Data Types 8.Register Data Types
Expressions Operands Operators
Modules and Ports Designing a Test Bench for Simulation
Built-In Primitives Built-In Primitives Design Examples
User-Defined Primitives Defining a User-Defined Primitive Combinational User-Defined Primitives Sequential User-Defined Primitives
Dataflow Modeling Continuous Assignment Reduction Operators Conditional Operators Relational Operators Logical Operators Bitwise Operators Shift Operators
Behavioral Modeling Initial Statement Always Statement Intrastatement Delay Interstatement Delay Blocking Assignments Nonblocking Assignments Conditional Statements Case Statement Loop Statements Logical, Algebraic, and Rotate Shift Operations
Structural Modeling Module Instantiation Ports Design Examples
Tasks and Functions

Task Declaration Task Invocation Function Declaration Function Invocation
Problems

Unit 1: Introduction to Logic Design Using Verilog

1.HDL:

In this provides an introduction to the **design methodologies** and **modeling** constructs of the Verilog hardware description language (HDL). Modules, ports, and test benches will be presented.

A hardware description language (HDL) is a programming language used to describe the behavior **or** structure of digital circuits (ICs). HDLs are also used to **stimulate the circuit** and check its response.

Program:

```
module my_design;
  initial
   begin
    $display("Hello Verilog world");
  end
endmodule
```

```
# run 1000ns
Hello Verilog world
```

Module . . . **endmodule**	Fundamental descriptive unit Building block of digital circuit
initial	Initial blocks are primily used for initialize variabes

begin **. . .** **end**	1.The **begin . . . end** keywords are used to **group** multiple statements into sequential blocks. The statements in a sequential block execute in sequence; that is, a statement does not execute until the preceding statement has executed, except for nonblocking statements. 2.If there is only one procedural statement in the block, then the **begin . . . end** keywords may be omitted.

For single statement: **module my_design;** **initial** **$display("Hello Verilog world");**	**For multiple statements**: **module my_design;** **initial** **$display("Hello Verilog world");**

endmodule	$display("Battula suresh"); endmodule
# run 1000ns Hello Verilog world	error

$display $write	This are display argments

```
module my_design;
initial
 begin
     $display("Hello Verilog world");
     $write("Hello world");
     $write ("Battula");
     $display("suresh");
end
endmodule
```

```
# run 1000ns
Hello Verilog world
Hello worldBattulasuresh
```

```
module my_design;
initial
begin
   $display ("10");
   $write ("20");
   $display ("30");
   $write ("40");
   $display ("50");
end
endmodule
```

```
Answer:
10
2030
4050
```

module my_design; initial begin	

$display("Hello Verilog world"); $write("Hello world\n"); $write ("Battula\t"); $display("suresh"); end endmodule	

2.Logic Elements

Logic elements are the constituent parts of the Verilog language. They consist of

comments,

logic gates,

parameters,

procedural control statements

which modify the flow of control in a behavior, and data types.

Lexical Conventions:

- Whitespaces
- Comments
- Operators
- Number Specification
- Strings
- Identifiers & Keywords

Whitespaces:

- "\t" (Tabs)
- "\n" (Newlines)

module my_design; **initial** **begin** **$display("Hello, World");** **end** **endmodule**
 # run 1000ns **Hello, World**

module my_design; **initial** **begin**	**Answer:** **numbers starts:** **10**

`$display ("numbers starts:");`
`$display ("10");`
`$write("20\n");`
`$display ("30");`
`$write ("40\n");`
`$display ("50");`
`$display ("numbers end");`
`$display ("thank you");`
`end`
`endmodule`
	20
	30
	40
	50
	numbers end
	thank you

`module my_design;`
`initial`
`begin`
`$display ("numbers starts:");`
`$display ("10");`
`$write("20\t");`
`$display ("30");`
`$write ("40\t");`
`$display ("50");`
`$display ("numbers end");`
`$display ("thank you");`
`end`
`endmodule`
	Answer:
	numbers starts:
	10
	20 30
	40 50
	numbers end
	thank you

`module my_design;`
`initial`
`begin`
`$display("numbers starts:");`
`$display("10");`
`$write ("20%%");`
`$display ("30");`
`$write ("40\\");`
`$display ("50\"");`
`$display ("numbers end");`
`$display ("thank you");`
`end`
`endmodule`
	Answer:
	numbers starts:
	10
	20%30
	40\50"
	numbers end
	thank you

Comments:

Two types:

1. single-line comment.

2. multi-line comment.

Ex: 1. // single line comment

2. /* this is a multiple line comment */

```
//This is a single-line comment on a dedicated line

assign z1 = x1 | x2//This is a comment on a line of code
```

```
/*This is a multiple-line comment.

   More comments go here.
   More comments. */
```

```
module my_design;
initial
   begin
    $display("Hello Verilog world");     //introduction
    $write("Hello world");
    $write ("Battula");          // my name starting
    $display("suresh");
    end
endmodule
```

```
# run 1000ns
Hello Verilog world
Hello worldBattulasuresh
```

```
module my_design;
initial
 begin
 $display("Hello Verilog world");//introduction
 $write("Hello world");
 $write ("Battula"); // my name starting
 $display("suresh");
end
endmodule
```

```
# run 1000ns
Hello Verilog world
Hello worldBattulasuresh
INFO: [USF-XSim-96] XSim completed. Design snapshot 'my_design_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
```

```
module my_design;
initial
   begin
     $display("Hello Verilog world"); //introduction
     $write("Hello world");
     /* not print any thing
     $write ("Battula"); // my name starting
     $display("suresh");
     */
    end
endmodule
```

```
# run 1000ns
Hello Verilog world
Hello worldINFO: [USF-XSim-96] XSim completed. Design snapshot 'my_design_behav' loaded.
```

```
module my_design;
initial
 begin

 $display("Hello Verilog world");//introduction
 $write("Hello world");
 /* not print any thing
 $write ("Battula"); // my name starting
 $display("suresh");
*/
end
endmodule
```

```
# run 1000ns
Hello Verilog world
Hello worldINFO: [USF-XSim-96] XSim completed. Design snapshot 'my_design_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
```

Number:

```
module my_design;

integer i=15;

initial
   begin
```

```
      $display("%d",i);
      $display("number=%d",i);


   end
endmodule
```

```
# run 1000ns
         15
number=         15
```

To avoid spacing and zeros:

```
module my_design;

integer i=15;

initial
   begin
      $display("%0d",i);
      $display("number=%0d",i);


   end
endmodule
```

```
# run 1000ns
15
number=15
```

%d =decimal

%b=binary

```
module my_design;

integer i=15;

initial
  begin
    $display("%0b",i);
    $display("number=%0d",i);


  end
endmodule
```

```
# run 1000ns
1111
```

number=15

Addition of two number:

```
module my_design;

integer i1=15;
integer i2=30;
integer i3;

initial
   begin
   i3=i1+i2;

     $display("i1=%0d",i1);
     $display("i2=%0d",i2);
     $display("i3=%0d",i3);

   end
endmodule
```

```
# run 1000ns
i1=15
i2=30
i3=45
```

```
module my_design;

integer i1=15,i2=30,i3;
//integer i2=30;
//integer i3;

initial
   begin
   i3=i1+i2;//expression inside begin end only

     $display("i1=%0d",i1);
     $display("i2=%0d",i2);
     $display("i3=%0d",i3);

   end
endmodule
```

```
# run 1000ns
i1=15
i2=30
i3=45
```

Number systems:

Binary	b or B
Octal	o or O
Decimal	d or D
Hexadecimal	h or H

```
module my_design;

integer i=15;

initial
   begin

      $display("decimal=%0d",i);
      $display("binary=%0b",i);
      $display("octal=%0o",i);
      $display("hexadecimal=%0h",i);

   end
endmodule
```

```
# run 1000ns
decimal=15
binary=1111
octal=17
hexadecimal=f
```

Number Specification:

They are two types

1. Sized Numbers

2. Un-sized Numbers

module my_design; **integer i=15;** **initial** **begin**	**# run 1000ns** **decimal: 15** **binary:1111** **octal:17**

$display ("decimal:%d",i);** **$display ("binary:%0.1b",i); **$display ("octal:%0.1o",i);** **$display ("hexadecimal:%0.1h",i);** **$display ("thank you");** **end** **endmodule**	**hexadecimal:f** **thank you**

Number Specification:

1. Sized Numbers:

Syntax: <size'><baseformat><number>

1'b1 logic 1

8'h21 8-bit number 0010_0001

12'ha35 12-bit number 101000110101

12'042xx 12-bit number 100010xxx xxx

4'd1 4-bit 0001

3s signed number

4'b1111 => this is a 4-bit binary number.

12'habc => this is a 12-bit hexadecimal number.

16'd225 => this is a 16-bit decimal number.

12'h13x => this is a 12-bit hexadecimal number.

```
module my_design;
integer i1=1'b1;          //1 bit  number
integer i2=2'b11;       //2 bit  number
integer i3=3'b111;     //3 bit  number
integer i4=4'b1111;     //4 bit  number
integer i5=8'b1111_1111;     //8 bit  number

    initial
        begin
        $display("i1=%0d  i1=%0b",i1,i1);
        $display("i2=%0d  i2=%0b",i2,i2);
        $display("i3=%0d  i3=%0b",i3,i3);
        $display("i4=%0d  i4=%0b",i4,i4);
        $display("i5=%0d  i5=%0b",i5,i5);

        end
```

```
endmodule
```

```
# run 1000ns
i1=1  i1=1
i2=3  i2=11
i3=7  i3=111
i4=15  i4=1111
i5=255  i5=11111111
```

```
module my_design;
integer i1=1'b1;    //1 bit  number
   initial
      begin
      $display("i1=%d",i1);
      $write("i1=%b\n",i1);
      end
endmodule
```

```
# run 1000ns
i1=        1
i1=00000000000000000000000000000001
```

```
module my_design;
integer i1=1'b1;            //1 bit  number
integer i2=2'b11;           //2 bit  number
   initial
      begin
      $display("i1=%d",i1);
      $write("i1=%b\n",i1);
      $display("i2=%d",i2);
      $write("i2=%b\n",i2);
      end
endmodule
```

```
# run 1000ns
i1=        1
i1=00000000000000000000000000000001
i2=        3
i2=00000000000000000000000000000011
```

```
module my_design;
integer i1=1'b1;        //1 bit  number
integer i2=2'b11;     //2 bit  number
integer i3=3'b111;    //3 bit  number
   initial
      begin
      $display("i1=%d",i1);
```

```
        $write("i1=%b\n",i1);
        $display("i2=%d",i2);
        $write("i2=%b\n",i2);
        $display("i3=%d",i3);
        $write("i3=%b\n",i3);
        end
endmodule
```

```
# run 1000ns
i1=         1
i1=00000000000000000000000000000001
i2=         3
i2=00000000000000000000000000000011
i3=         7
i3=00000000000000000000000000000111
```

```
module my_design;
integer i=10;
   initial
        begin
        $display("%d",i);
        $write("%b\n",i);
        end
endmodule
```

```
# run 1000ns
        10
00000000000000000000000000001010
```

```
module my_design;
integer i=8'b0000_0101;
integer i2=4'b0010;
   initial
        begin
        $display("%d",i);
        $write("%b\n",i);
        $display("%d",i2);
        $write("%b\n",i2);
        end
endmodule
```

```
# run 1000ns
         5
00000000000000000000000000000101
```

```
        2
00000000000000000000000000000010
```

```
module my_design;
integer i=8'b0000_0101;
integer i2=4'b0010;
   initial
      begin
      $display("%d",i);
      $write("%0.1b\n",i);
      $display("%d",i2);
      $write("%0.1b\n",i2);
      end
endmodule
```

```
# run 1000ns
        5
101
        2
10
```

```
module my_design;
integer i1=4'b1111;
integer i2=8'b0000_1111;
integer i3=12'habc;
integer i4=16'd255;
   initial
      begin
      $display("%d",i1);
      $write("%0.1b\n",i1);
      $display("%d",i2);
      $write("%0.1b\n",i2);
      $display("%d",i3);
      $write("%0.1b\n",i3);
      $display("%d",i4);
      $write("%0.1b\n",i4);
      end
endmodule
```

```
# run 1000ns
```

```
      15
1111
      15
1111
    2748
101010111100
     255
11111111
```

```
module my_design;
integer i1=8'b00011000; //1 bit  number
   initial
      begin
      $display("i1=%d",i1);
      $write("i1=%0.1b\n",i1);
      end
endmodule
```

```
# run 1000ns
i1=      24
i1=11000
```

2.Un-Sized Numbers:

Syntax: '<baseformat><number>

'b1111 => this is a 32-bit binary number

'O21 => this is a 32-bit Octal number

'd225 => this is a 32-bit decimal number

'h13x => this is a 32-bit hexadecimal number

```
module my_design;
integer i1=23456;          //32-bit decimal number
integer i2='hc3;           //32-bit hexadecimal number
integer i3='o21;           //32-bit octal number
   initial
      begin
      $display(" %d",i1);
      $write(" %0.1b\n",i1);
      $display(" %d",i2);
```

```
        $write(" %0.1b\n",i2);
        $display(" %d",i3);
        $write(" %0.1b\n",i3);
        end
endmodule
```

```
# run 1000ns
     23456
101101110100000
       195
11000011
        17
10001
```

X or Z values:

```
module my_design;
integer i1=12'h13x;    //2-bit decimal number
integer i2=6'hx;       //32-bit hexa decimal number
integer i3=32'bz;      //32-bit octal number
   initial
      begin
      $display("i1=%d",i1);
      $write("i1=%0.1b\n",i1);
      $display("i2=%d",i2);
      $write("i2=%0.1b\n",i2);
      $display("i3=%d",i3);
      $write("i3=%0.1b\n",i3);
      end
endmodule
```

```
# run 1000ns
i1=        X
i1=10011xxxx
i2=        X
i2=xxxxxx
i3=        z
i3=zzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzz
```

Negative numbers:

```
module my_design;
integer i1=-6'd3;        //8 bit negative number stored in 2's cpmplement of 3
//integer i2=4'd-2;      //illegal specification
   initial
      begin
      $display("i1=%d",i1);
      $write("i1=%0.1b\n",i1);
      end
endmodule
```

```
# run 1000ns
i1=        -3
i1=11111111111111111111111111111101
```

Verilog format specifiers:

%h %H	Display in **hexadecimal** format
%d %D	Display in **decimal** format
%b %B	Display in **binary** format
%m %M	Display in **hierarchical** name
%s %S	Display as a **string**
%t %T	Display in **time** format
%f %F	Display **real** in decimal format
%e %E	Display in a **exponential** format

Strings :

Strings can be stored in register.

The width of the register variables must be large enough to hold the strings.

If the width of the register is greater than the size of the string ,Verilog fills bits to the left of the string with zeros.

- Group of chars within double quotes.
- Do not embed newlines.
- It can't be on multiple lines.
- Each Character in the string requires **1Byte** to store.
- You can use escaped characters (such as \t and \n) in strings.

Examples:

“Hello Verilog World” => is a string

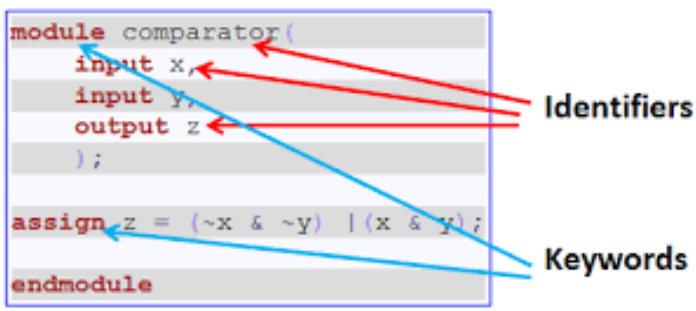
“a /b ” => is a string

```
module my_design;
reg [256:0] str="hello verilog world";
   initial
      begin
      $display("str=%s",str);
      end
endmodule
```

```
# run 1000ns
str=            hello verilog world
```

Identifiers :

- Identifiers are names of variables so that they can be referenced later on.
- They are made up of alphanumeric characters [a-z][A-Z][0-9] , underscores _ or dollar sign $ and are case sensitive. They cannot start with a **digit** or a **dollar sign.**



Data types:

1.Data Types in Verilog

A storage format having a **specific range** or **type** is called data type. They can be divided into two groups

i.Net type group: The net-type group represents physical connections between digital circuits.

Ex: wire, wand, wor, etc

ii. Variable type group: The variable type group represents the storage of values in digital circuits. They are used as variables.

Ex: reg, integer

'**?**' is the same as '**z**' (only regarding numbers) **4'b10??** => the same as **4'b10zz**

Value levels	Condition in hardware circuits
0	Logic zero , False condition
1	Logic one, True condition
X	Unknown value
Z	High impedance, floating state

Value	Name	Description	Hardware Interpretation	Common Use Case
`0`	Logic Zero, False	Represents a false or low state.	Low voltage (e.g., 0V).	False in Boolean logic, inactive signal.
`1`	Logic One, True	Represents a true or high state.	High voltage (e.g., 3.3V, 5V).	True in Boolean logic, active signal.
`X`	Unknown	Represents an unknown, indeterminate, or conflicting value.	Undefined or conflicting state.	Signal conflicts, uninitialized state in simulations.
`Z`	High Impedance, Floating	Represents a high impedance (floating) state where no value is driven.	No voltage; signal is neither high nor low.	Tri-state buffers, shared buses, open-drain/open-collector outputs.

Register:

- Register represents data storage elements.
- It is a variable that can hold a value.
- It s commonly declared by keyword '**reg**'.
- The default value of reg data type is **x**(unknown).
- Register represents a class of **data types** such as

  1.reg (most wisely used),
  2.integer (used for loop counting),
  3.real (used to store floating point numbers),
  4.time (keep track of simulation time).

Vectors:

- **Net** or **reg** data types can be declared as vectors (multiple bit widths).
- If bit width is not specified, the default is scalar (1-bit)

Wire a ; //scalar net variable ,default

Wire [7:0] bus; // 8-bit bus

Wire [31:0] bus A, bus B, bus C; // 3 buses of 32-bit width

Vectors can be declared at [high #: low #] or [low# :high #]

Arrays:

- In Verilog arrays are allowed for reg, integer, time, real, vector, register data types.

<array name> [<subscript>]

- Multidimensional arrays are not allowed in Verilog.

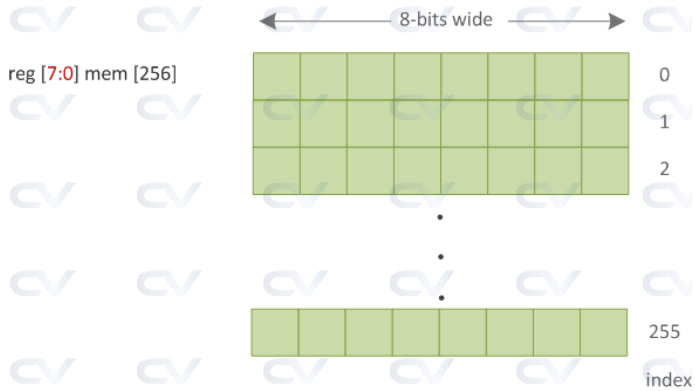
```
module my_design;
integer b=4'b1111;  //integer
wire [7:0] a;  // vector
reg num[31:0];// array of vectors
reg [7:0] mem[255:0];   // memory
//integer b=4'b1111;  //integer
   initial
      begin
      $display("a=%d",a);
      $display("a=%b",a);
      $display("b=%d",b);
      $display("b=%b",b);
      //$display("mem=%b",mem);
      end
endmodule
```

```
# run 1000ns
a=  z
a=zzzzzzzz
b=        15
b=00000000000000000000000000001111
```

Memories:

- Memories are modeled as 1D array of **register.**
- Used to model register flies, ROM's and RAMS's.
- Each element of the array is known as element/word.

Reg [7:0] mem[256] // memory 8 bit wide , 256 address



Memories can be represented in Verilog by an array of registers and are declared using a reg data type as follows:

Number of bits per register ↓ Number of registers ↓

reg [msb:lsb] memory_name [first address:last address];

An array can have only two dimensions. **Memories** must be declared as **reg** data types, not as wire data types. A register can be assigned a value using one statement, as shown below.

A **32-word register** with **one byte per word** (A byte is eight bits) would be declared as follows:

reg [7:0] memory_name [0:31];

```
module my_design;
reg [7:0] memory_name[0:31];
   initial
      begin
      //$display("memory_name =%b",memory_name);
      //$display("memory_name =%d",memory_name);
      end
endmodule
```

Memory locations not find

Register **buff_reg** is assigned the 16-bit hexadecimal value of 7ab5, which equates to the binary value of $0111\ 1010\ 1011\ 0101_2$.

reg [15:0] buff_reg;

buff_reg = 16'h7ab5;

```
module my_design;
reg [15:0] buff_reg=16'h7ab5;
   initial
```

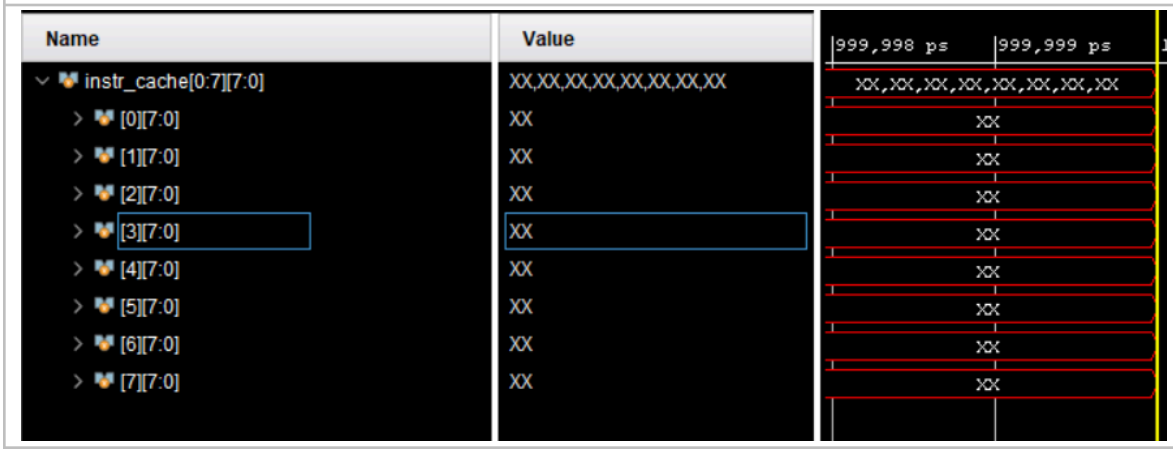
```
        begin
        $display("buff_reg=%b",buff_reg);
        $display("buff_reg=%d",buff_reg);
        end
endmodule
```

```
# run 1000ns
buff_reg=0111101010110101
buff_reg=31413
```

Values can also be stored in memories by assigning a value to each word individually, as shown below for an instruction cache of **eight registers** with **eight bits** per register.

reg [7:0] instr_cache [0:7];

```
module my_design;
reg [7:0] instr_cache [0:7];
    initial
        begin
        //$display("memory_name =%b",memory_name);
        //$display("memory_name =%d",memory_name);
        end
endmodule
```



System tasks :

$monitor

$stop

$finish

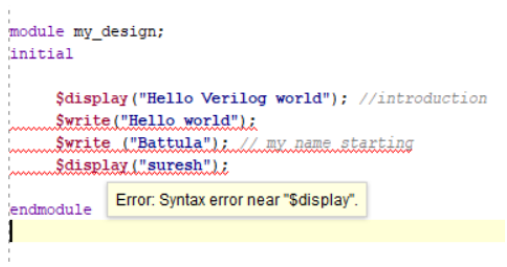
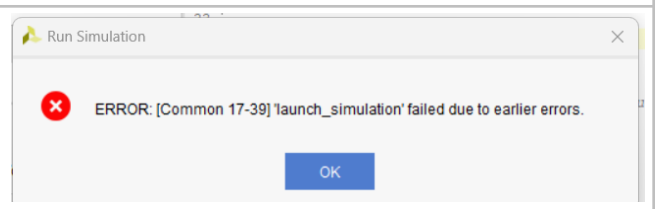
‘define

‘timescale

Procedural Flow Control:

Procedural flow control statements modify the flow in a behavior by selecting branch options, repeating certain activities, selecting a parallel activity, or terminating an activity. The activity can occur in sequential blocks or in parallel blocks.

begin **. . .** **end**	1.The **begin . . . end** keywords are used to **group** multiple statements into sequential blocks. The statements in a sequential block execute in sequence; that is, a statement does not execute until the preceding statement has executed, except for nonblocking statements. 2.If there is only one procedural statement in the block, then the **begin . . . end** keywords may be omitted.

module my_design; **initial** **$display ("**Hello Verilog world**"); //introduction** **endmodule**	# run 1000ns Hello Verilog world
	

module my_design;
initial
begin
$display ("Hello Verilog world**"); //introduction**
$write ("Hello world**");**
$write ("Battula**"); // my name starting**
$display("suresh**");**
end
endmodule
	# run 1000ns
	Hello Verilog world
	Hello worldBattulasuresh

disable	The disable statement terminates a named block of procedural statements or a task and transfers control to the statement immediately following the block or task. The disable statement can also be used to exit a loop.

1.for loop

for	The keyword **for** is used to specify a loop. The for loop repeats the execution of a procedural statement or a block of procedural statements a specified

	number of times. The for loop is used when there is a specified beginning and end to the loop. The format and function of a for loop is similar to the for loop used in the **C programming** language. The parentheses following the keyword for contain three expressions separated by semicolons, as shown below.

```
for(initialization; condtion; update)
begin
        .....
end
```

module my_design; **integer i;** **initial** **begin** **for(i=0;i<10;i=i+1)** **begin** **$display("i=%d",i);** **end** **end** **endmodule**	**# run 1000ns** **i= 0** **i= 1** **i= 2** **i= 3** **i= 4** **i= 5** **i= 6** **i= 7** **i= 8** **i= 9**

Display 1 to 15 binary and decimal numbers

`module my_design;`
`integer i=1;`
 	
`initial`
`for(i=1;i<=15;i=i+1)`
`begin`
`$display("%0d=%0b",i,i);`
`end`
 	
`endmodule`
	`# run 1000ns`
	`1=1`
	`2=10`
	`3=11`
	`4=100`
	`5=101`
	`6=110`
	`7=111`
	`8=1000`
	`9=1001`
	`10=1010`
	`11=1011`
	`12=1100`
	`13=1101`

	14=1110 15=1111

2.while loop

while	The **while** statement executes a statement or a block of statements while an expression is true.

```
module my_design;
integer i=5;
initial
begin
while (i>0)
begin
$display("i=%d",i);
i=i-1;
end
end
endmodule
```

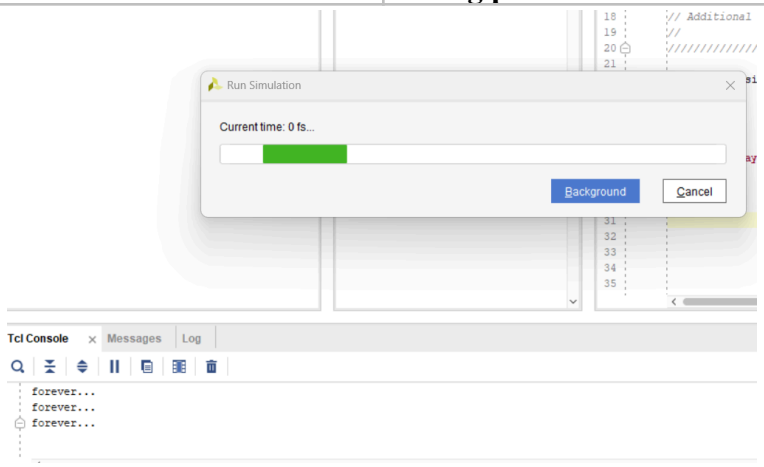
```
# run 1000ns
i=      5
i=      4
i=      3
i=      2
i=      1
```

3.forever loop:

forever	The **forever loop** statement executes the **procedural statements continuously**. The loop is primarily used for timing control constructs, such as **clock pulse generation**. The forever procedural statement must be contained within an **initial** or an **always** block. In order to exit the loop, the **disable** statement may be used to prematurely terminate the procedural statements. An always statement executes at the beginning of simulation; the forever statement executes only when it is encountered in a procedural block

```
forever
  statements
forever: begin
  multiple statements
end
```

module my_design; **initial** **begin** **forever** **begin** **$display("forever...");** **end** **end** **endmodule**	**forever...** **forever...** **forever...** **forever...** **forever...** **forever...** **forever...** **forever...** **……..** **killing process…**

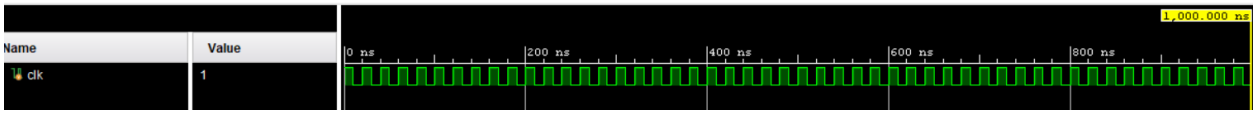


Clock generation:

```
module my_design;
   reg clk;  // Declare the clock signal

   // Initial block to generate clock pulse
   initial
   begin
      clk = 0;  // Set initial clock value to 0
      forever
      begin
         clk = ~clk;  // Toggle the clock every 5 time units (simulates a clock pulse)
         #10 $display($time,"ns Clock Pulse: %b", clk);
      end
   end
endmodule
```

```
# run 1000ns
              10ns Clock Pulse: 1
```

```
20ns Clock Pulse: 0
30ns Clock Pulse: 1
40ns Clock Pulse: 0
50ns Clock Pulse: 1
60ns Clock Pulse: 0
.
.
.
1000ns Clock Pulse: 0
```



4.repeat

repeat	The **repeat** keyword is used to execute a loop a fixed number of times as specified by a constant contained within parentheses following the repeat keyword. The loop can be a single statement or a block of statements contained within **begin . . . end** keywords. The syntax is shown below.

repeat (number of times) **begin** **statements** **end**	**repeat (number of times)@(some event)** **begin** **statements** **end**

```
module my_design;
    initial
    begin
      repeat (4)
      begin
      $display ("hello, I am suresh");
      end
    end
endmodule
```

```
hello, I am suresh
hello, I am suresh
hello, I am suresh
hello, I am suresh
```

Feature	`forever`	`repeat`
Termination	Runs indefinitely until simulation ends	Runs a fixed number of times (specified by the user)
Use Case	Used for continuous behavior (e.g., clock generation, stimulus)	Used when a finite number of repetitions is needed
Syntax	`forever begin ... end`	`repeat (N) begin ... end`

5.if . . . else These keywords are used as conditional statements to alter the flow of activity through a behavioral module. They permit a choice of alternative paths based upon a Boolean value obtained from a condition.

The syntax is shown below.

```
if (condition)
{procedural statement 1}
 else
 {procedural statement 2}
```

5.1 if with out else for single statement:

```
module my_design;
  integer a=10;
  initial
    begin
     if(a==10)
     $display("a is equal to 10");
     $display("executed completely");
    end
endmodule
```

```
run 1000ns
a is equal to 10
executed completely
```

5.2 if without else for multiple statement:

```
module my_design;
  integer a=10;
  initial
    begin
     if(a==10)
```

```
      begin
      $display("a is equal to 10");
      $display("executed completely");
     end
     $display("out side loop");
    end
endmodule
```

```
#run 1000ns

a is equal to 10
executed completely
out side loop
```

5.3 if-else for single statement:

```
module my_design;
   integer a=19;
   initial
     begin
      if(a==10)
      $display("a is equal to 10");
     else
     $display("a is not equal to 10");
    end
endmodule
```

```
# run 1000ns
a is not equal to 10
```

5.4 if-else for multiple statement:

```
module my_design;
   integer a=19;
   initial
     begin
       if(a==10)
       begin
       $display("a is equal to 10");
       end
       else
       begin
       $display("a is not equal to 10");
       $display("reason for a is not equal to 10");
       end
   end
endmodule
```

```
# run 1000ns
a is not equal to 10
reason for a is not equal to 10
```

Operators:

Types:

1. unary operator. a = ~ b;

2. Binary operator. a = b & c;

3. Ternary operator. a = b? c: d;

1.Unary operator:

```
module my_design;
integer b=4'b1111;
integer a;

initial
begin
a=~b;
$display("%b",b);
$display("%b",a);

end
endmodule
```

```
# run 1000ns
00000000000000000000000000001111
00000000000000000000000000000000
```

```
module my_design;
integer b=4'b1111;
integer a;

initial
begin
a=&b;
$display("%0b",b);
$display("%0b",a);

end
endmodule
```

```
# run 1000ns
1111
0
```

2.Binary operator

```
module my_design;
integer b=4'b1111;
integer c=4'b0001;
integer a;

initial
begin
a=b&c;
$display("%b",b);
$display("%b",c);
$display("%b",a);

end
endmodule
```

```
# run 1000ns
00000000000000000000000000001111
00000000000000000000000000000001
00000000000000000000000000000001
```

```
module my_design;
integer b=4'b1111;
integer c=4'b0001;
integer a;
initial
begin
a=b&c;
$display("%0b",b);
$display("%0b",c);
$display("%0b",a);
end
endmodule
```

```
# run 1000ns
1111
1
1
```

3.Ternary operator

```
module my_design;
integer b=2;
integer c=3;
```

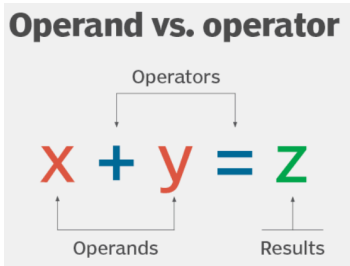
```
integer a;

initial
begin
a=(b>c)?b:c;
$display("a=%d",b);
$display("c=%d",c);
$display("a=%d",a);

end
endmodule
```

```
# run 1000ns
a=        2
c=        3
a=        3
```

II.Expressions

Operands

Operators



Operators:

Verilog HDL contains a profuse set of operators that perform various operations on different types of data to yield results on nets and registers. Some operators are similar to those used in the C programming language. Table 1 lists the categories of operators in order of precedence, from highest to lowest.

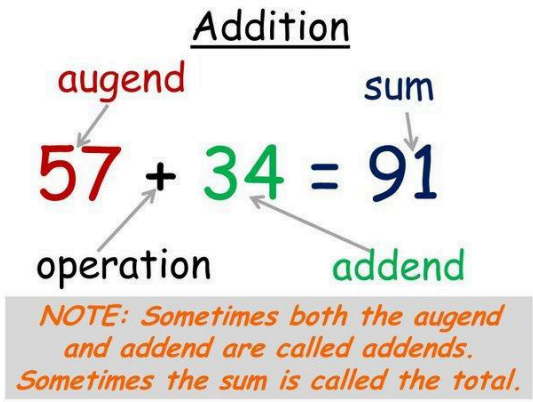
Operator type	Operator Symbol	Operation	Number of Operands
Arithmetic	+	Add	Two or one

	-	Subtract	Two or one
	*	Multiply	Two
	/	Divide	Two
	%	Modulus	Two
Logical	&&	Logical AND	Two
	||	Logical OR	Two
	!	Logical negation	One
Relational	>	Greater than	Two
	<	Less than	Two
	>=	Greater than or equal	Two
	<=	Less than or equal	Two
Equality	= =	Logical equality	Two
	! =	Logical inequality	Two
	= = =	Case equality	Two
	! = =	Case inequality	Two
Bitwise	&	AND	Two
	|	OR	Two
	~	Negation	One
	^	Exclusive-OR	Two
	^ ~ or ~ ^	Exclusive-NOR	Two
Reduction	&	AND	One
	~ &	NAND	One
	|	OR	One
	~ |	NOR	One
	^	Exclusive-OR	One
	~ ^ or ^ ~	Exclusive-NOR	One
Shift	<<	Left shift	One
	>>	Right shift	One
Conditional	? :	Conditional	Three
Concatenation	{ }	Concatenation	Two or more
Replication	{{ }}	Replication	Two or more

Arithmetic:

Arithmetic operations are performed on **one (unary**) operand or **two** (binary) operands in the following radices: **binary, octal, decimal**, or **hexadecimal**. The result of an arithmetic operation is interpreted as an unsigned value or as a signed value in 2s complement representation on both scalar and vector **nets** and **registers**.

The operands shown in **Table 1** are used for the operations of **addition, subtraction, multiplication, and division.**



module my_design; **integer i1;** **integer i2;** **integer i3;** **initial** **begin** **i1=15;** **i2=10;** **i3=i1+i2;** **$display ("Arithmetic operations:");** **$display ("%d",i3);** **end** **endmodule**	**# run 1000ns** **Arithmetic operations:** **25**

```
//////////////////////////////////////////

module my_design;
integer i1=15;
integer i2=10;
integer i3=i1+i2;
    initial
    begin
    //i1=15;
    //i2=10;
    //i3=i1+i2;
    $display ("Arithmetic operations:");
    $display ("%d",i3);
    end
endmodule
```

Error: i1 is not a constant
Error: i2 is not a constant

module my_design; **integer i1=15;** **integer i2=10;** **integer i3;** **initial** **begin**	**# run 1000ns** **Arithmetic operations:** **25**

```
    i3=i1+i2;
    $display ("Arithmetic operations:");
    $display ("%d",i3);
    end
endmodule
```

```
module my_design;
integer i1;
integer i2;
integer i3;
integer i4;
integer i5;
integer i6;
integer i7;
    initial
    begin
    i1=15;
    i2=10;
    i3=i1+i2;
    i4=i1-i2;
    i5=i1/i2;
    i6=i1*i2;
    i7=i1%i2;
    $display ("Arithmetic operations:");
    $display ("addition:%d",i3);
    $display ("Subtraction:%d",i4);
    $display ("division:%d",i5);
    $display ("multiply:%d",i6);
    $display (" Modulus:%d",i7);
    end
endmodule
```

```
# run 1000ns
Arithmetic operations:
addition:       25
Subtraction:        5
division:       1
multiply:       150
 Modulus:       5
```

```
module my_design;
integer i1,i2,i3,i4,i5,i6,i7;
    initial
```

```
    begin
    i1=15;
    i2=10;
    i3=i1+i2;
    i4=i1-i2;
    i5=i1/i2;
    i6=i1*i2;
    i7=i1%i2;
    $display ("Arithmetic operations:");
    $display ("addition:%0.1d+%0.1d=%0.1d",i1,i2,i3);
    $display ("Subtraction:%0.1d-%0.1d=%0.1d",i1,i2,i4);
    $display ("division:%0.1d/%0.1d=%0.1d",i1,i2,i5);
    $display ("multiply:%0.1d*%0.1d=%0.1d",i1,i2,i6);
    $display ("Modulus %0.1d mod %0.1d=%d =%0.1d",i1,i2,i7);
    end
endmodule
```

```
# run 1000ns
Arithmetic operations:
addition:15+10=25
Subtraction:15-10=5
division:15/10=1
multiply:15*10=150
Modulus:15 mod 10=5
```

```
module my_design;
integer i1,i2,i3,i4,i5,i6,i7;
    initial
    begin
    i1=15;i2=10;i3=i1+i2;i4=i1-i2;i5=i1/i2;i6=i1*i2;i7=i1%i2;
    $display ("Arithmetic operations:");
    $display ("addition:%0.1d+%0.1d=%0.1d",i1,i2,i3);
    $display ("Subtraction:%0.1d-%0.1d=%0.1d",i1,i2,i4);
    $display ("division:%0.1d/%0.1d=%0.1d",i1,i2,i5);
    $display ("multiply:%0.1d*%0.1d=%0.1d",i1,i2,i6);
    $display ("Modulus %0.1d mod %0.1d=%d =%0.1d",i1,i2,i7);
    end
endmodule
```

```
# run 1000ns
Arithmetic operations:
addition:15+10=25
Subtraction:15-10=5
division:15/10=1
multiply:15*10=150
Modulus:15 mod 10=5
```

Net or **register** operands are treated as **unsigned** values;

real or **integer** operands are treated as signed values.

+45(Positive 45_{10})

–72(Negative 72_{10})

module my_design; **integer i1,i2,i3;** **initial** **begin** **i1=10;i2=-6'd3;i3=i1+i2;** **$display("%0d",i1);** **$display("%0d",i2);** **$display("%0d",i3);** **end** **endmodule**	**# run 1000ns** **10** **-3** **7**

module my_design; **reg [3:0] i1,i2,i3;** **initial** **begin** **i1=10;i2=-6'd3;i3=i1+i2;** **$display("%0d",i1);** **$display("%0d",i2);** **$display("%0d",i3);** **end** **endmodule**	**# run 1000ns** **10** **13** **7**

Reg size:

module my_design; **reg [3:0] i1,i2,i3;** **initial** **begin** **i1=5; i2=1; i3=i1+i2;** **$display("%0b %0d",i1,i1);** **$display("%0b %0d",i2,i2);** **$display("%0b %0d",i3,i3);** **end** **endmodule**	**module my_design;** **reg [3:0] i1,i2,i3;** **initial** **begin** **i1=5;i2=-'d1;i3=i1+i2;** **$display("%0b %0d",i1,i1);** **$display("%0b %0d",i2,i2);** **$display("%0b %0d",i3,i3);** **end** **endmodule**
# run 1000ns **101 5** **1 1** **110 6**	**# run 1000ns** **101 5** **1111 15** **100 4**

reg first define memory size other wise default size 1

module my_design; **reg i1,i2,i3;** **initial** **begin** **i1=5;i2=-'d1;i3=i1+i2;** **$display("%0b %0d",i1,i1);** **$display("%0b %0d",i2,i2);** **$display("%0b %0d",i3,i3);** **end** **endmodule**	**# run 1000ns** **1 1** **1 1** **0 0**

Logical operators deal with **statements**.

Bitwise operators deal with **binary** representations of data.

Logical:

There are three logical operators: the binary logical AND operator (**&&**),

the binary logical OR operator **(| |)**, and

the unary logical negation operator **(!).**

Logical operators evaluate to a **logical 1** (**true**), a **logical 0** (false), or an **x (ambiguous).**

If a logical operation **returns** a nonzero value, then it is treated as a **logical 1 (true);**

if a bit in an operand is **x or z**, then it is **ambiguous** and is normally treated as a false condition.

Let **a** and **b** be two 4-bit operands, where a = 0110 and b = 1100.

module my_design; **integer a=0110,b=0111;** **integer z1,z2,z3;** **initial** **begin** **z1=a&&b;z2=a|b;z3=!a;** **$display("%0d %0b",z1,z1);** **$display("%0d %0b" ,z2,z2);** **$display("%0d %0b",z3,z3);** **end** **endmodule**	**# run 1000ns** **1 1** **111 1101111** **0 0**

Specifically, the integer data type is meant for signed or unsigned 32-bit variables, but you're attempting to use them with binary values and logical operations.

module my_design;	# run 1000ns

integer a=15,b=10,c=5;
integer z1,z2,z3;
initial
begin
z1= a>b && a>c;
z2= a>b || a>c;
z3= !a;
 	
$display("%0b",z1);
$display("%0b",z2);
$display("%0b",z3);
end
endmodule
	1
	1
	0

module my_design;
reg [3:0] a,b;
reg [3:0] z1,z2,z3;
initial
begin
a=0111;b=1111;
z1=a&&b;z2=a|b;z3=!a;
$display("%0d %0b %0d %0b",a,a,b,b);
$display("%0d %0b",z1,z1);
$display("%0d %0b" ,z2,z2);
$display("%0d %0b",z3,z3);
end
endmodule
	# run 1000ns
	15 1111 7 111
	1 1
	15 1111
	0 0

module my_design;
reg [1:0] a,b;
reg [1:0] z1,z2,z3;
initial
begin
a=0101;b=0000;
z1=a&&b;z2=a|b;z3=!a;
$display("a=%0d a=%0b b=%0d b=%0b ",a,a,b,b);
$display("%0d %0b",z1,z1);
$display("%0d %0b" ,z2,z2);
$display("%0d %0b",z3,z3);
end
endmodule
	# run 1000ns
	a=1 a=1 b=0 b=0
	0 0
	1 1
	0 0

Relational:

Relational operators compare operands and return a Boolean result, either **1 (true)** or **0 (false)** indicating the relationship between the two operands.

There are four relational operators as follows: greater than (>),

less than (<),

greater than or eqaul(>=) and

less than or equal (<=).

These operators function the same as identical operators in the C programming language.

If the relationship is **true**, then the result is **1**; if the relationship is **false**, then the result is **0**.

Net or **register** operands are treated as **unsigned** values;

real or **integer** operands are treated as **signed** values.

An x or z in any operand returns a result of x.

When the operands are of **unequal size**, the smaller operand is **zero-extended** to the **left**.

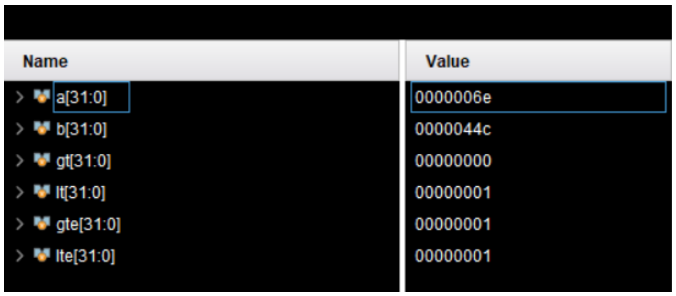
Examples:where the **identifier**

gt means **greater than**,
lt means **less than**,
gte means **greater than or equal**, and
lte means less than or equal

when comparing **operand a** to **operand b**.

```
module my_design;
integer a=4'b0110,b=4'b1100;
integer gt,lt,gte,lte;
   initial
   begin
   gt=a>b;
   lt=a<b;
   gte=  a>=b;
   lte=  a<=b;
   $display("%d  %d",a,b);
   $display("%0b  %0b",a,b);
   $display("gt=%0b %0b %0b %0b",gt,lt,gte,lte);
   end
endmodule
```

```
# run 1000ns
        6          12
110  1100
0 1 0 1
```



module my_design; **integer a=7,b=5;** **integer gt,lt,gte,lte;** **initial** **begin** **gt=a>b;** **lt=a<b;** **gte= a>=b;** **lte= a<=b;** **//$display("a=%0d b=%0d",a,b);** **$display("a=%0b b=%0b a=%0d b=%0d",a,b,a,b);** **$display("%0b %0b %0b %0b",gt,lt,gte,lte);** **end** **endmodule**	**# run 1000ns** **a=111 b=101 a=7 b=5** **1 0 0 0**

Equality:

There are **four** equality operators:

logical equality (= =),
logical inequality (**!** =),
case equality (= = =), and
case inequality (**!** = =).

Logical equality is used in expressions to determine if two values are **identical**. The result of the comparison is **1** if the two operands are **equal**, and **0** if they are **not equal**.
The **logical inequality** operator is used to determine if two operands are **unequal**. A **1** is returned if the operands are **unequal**; otherwise a **0** is returned.
The **case equality** operator compares both operands on a **bit-by-bit** basis, including x and z. The result is 1 if both operands are identical in the same bit positions, including those bit positions containing an x or a z.
The **case inequality** operator is used to determine if two operands are **unequal b**y comparing them on a bit-by-bit basis, including those bit positions that contain x or z.

If the result of the comparison is ambiguous for logical equality or logical inequality, then a value of **x** is returned. An **x** or **z** in either operand will return a value of **x**.

If the operands are nets or registers, they are treated as unsigned values; real or integer operands are treated as signed values, but are compared as though they were unsigned operands.

Example:

Where the 4-bit variables are x1 , x2 , x3 , x4 , and x5 . The outputs are z1 (logical equality), z2 (logical inequality), z3 (case equality), and z4 (case inequality).

```
I:   x1 = 1000, x2 = 1101, x3 = 01xz, x4 = ‘b01xz, x5 = ‘bx1xx
     z 1 = 0, z2 = 1, z3 = 1, z4 = 1
II:  x1 = 1011, x2 = 1011, x3 = ‘bx1xz, x4 = ‘bx1xz, x5 = ‘b11xx
     z1 = 1, z2 = 1, z3 = 1, z4 = 1
III:x1 = 1100, x2 = 0101, x3 = ‘bx01z, x4 = ‘b11xz, x5 = ‘b11xx
    z1 = 0, z2 = 1, z3 = 0, z4 = 1
```

I:

```
module my_design;
reg [7:0] X1=1000,X2=1101,X3='b01xz,X4='b01xz,X5 = 'bx1xx;
reg [7:0] z1,z2,z3,z4;
   initial
   begin
   z1=(X1 == X2);
   z2=(X2 != X3);
   z3=(X3 === X4);
   z4=(X4 !== X5);
   $display("z1=%0d z2=%0d z3=%0d z4=%0d",z1,z2,z3,z4);
   end
endmodule
```

```
# run 1000ns
z1=0 z2=1 z3=1 z4=1
```

Bitwise:

The bitwise operators are: AND (&),

OR (|),

negation (~),

exclusive-OR (^), and

exclusive-NOR (^ ~ or ~ ^).

The **bitwise** operators perform **logical** operations on the operands on a **bit-by-bit** basis and produce a **vector** result.

Except for **negation**, each bit in one operand is associated with the corresponding bit in the other operand. If one operand is shorter, then it is **zero-extended** to the left to match the length of the longer operand.

The **bitwise AND** operator performs the AND function on two operands on a bit-by-bit basis as shown in the following example	1 0 1 1 0 1 1 0 &) 1 1 0 1 0 1 0 1 1 0 0 1 0 1 0 0
The **bitwise OR** operator performs the OR function on the two operands on a bit-by-bit basis as shown in the following example:	1 0 1 1 0 1 1 0 |) 1 1 0 1 0 1 0 1 1 1 1 1 0 1 1 1
The **bitwise negation** operator performs the negation function on one operand on a bit-by-bit basis. Each bit in the operand is inverted as shown in the following example:	~) 1 1 0 1 0 1 0 1 0 0 1 0 1 0 1 0
The **bitwise exclusive-OR** operator performs the exclusive-OR function on two operands on a bit-by-bit basis as shown in the following example:	1 0 1 1 0 1 1 0 ^) 1 1 0 1 0 1 0 1 0 1 1 0 0 0 1 1
The **bitwise exclusive-NOR** operator performs the exclusive-NOR function on two operands on a bit-by-bit basis as shown in the following example:	1 0 1 1 0 1 1 0 ^~) 1 1 0 1 0 1 0 1 1 0 0 1 1 1 0 0

module my_design; **reg [7:0] X1,X2;** **reg [7:0] z1;** **initial** **begin** **X1=8'b11000011;** **X2=8'b10011001;**	**# run 1000ns** **X1=11000011** **X2=10011001** **z1=10000001**

z1=(X1&X2); $display("X1=%b",X1); $display("X2=%b",X2); $display("z1=%b",z1); end endmodule	

module my_design;
 	
reg [7:0] X1,X2;
reg [7:0] z1;
 	
initial
begin
X1=8'b1100_0011;
X2=8'b1001_1001;
z1=(X1&X2);
$display("X1=%b",X1);
$display("X2=%b",X2);
$display("z1=%b",z1);
end
endmodule
	# run 1000ns
	X1=11000011
	X2=10011001
	z1=10000001

a = 11000011, b = 10011001, and_rslt = 10000001, or_rslt = 11011011, neg_rslt = 00111100, xor_rslt = 01011010, xnor_rslt = 10100101	a = 01001111, b = 11011001, and_rslt = 01001001, or_rslt = 11011111, neg_rslt = 10110000, xor_rslt = 10010110, xnor_rslt = 01101001
a = 10010011, b = 11011001, and_rslt = 10010001, or_rslt = 11011011, neg_rslt = 01101100, xor_rslt = 01001010, xnor_rslt =10110101	a = 11001111, b = 11011001, and_rslt = 11001001, or_rslt = 11011111, neg_rslt = 00110000, xor_rslt = 00010110, xnor_rslt = 11101001

```
module my_design;

reg [7:0] a,b;
reg [7:0]  and_rslt,or_rslt,neg_rslt,xor_rslt,xnor_rslt;

   initial
   begin
   a=8'b1100_0011;
   b=8'b1001_1001;

   and_rslt=(a&b);
   or_rslt=(a|b);
   neg_rslt=(~a);
   xor_rslt=(a^b);
   xnor_rslt=(a^~b);
   $display("a=%b",a);
   $display("b=%b",b);
   $display("and_rslt=%b",and_rslt);
   $display("or_rslt=%b",or_rslt);
   $display("neg_rslt=%b",neg_rslt);
   $display("xor_rslt=%b",xor_rslt);
    $display("xnor_rslt=%b",xnor_rslt);
   end
endmodule
```

```
# run 1000ns
a=11000011
b=10011001
and_rslt=10000001
or_rslt=11011011
neg_rslt=00111100
xor_rslt=01011010
xnor_rslt=10100101
```

Reduction:

The reduction operators are: AND (&),

NAND (~&),

OR (|),

NOR (~ |),

exclusive-OR (^), and

exclusive-NOR (^ ~ or ~ ^).

Reduction operators are **unary operators**; that is, they operate on a **single vector** and produce a **single-bit result**. If any bit of the operand is **x** or **z**, the result is **x**. Reduction operators perform their respective operations on a **bit-by-bit** basis.

For the reduction **AND operator**, if any bit in the operand is 0, then the result is 0; otherwise, the result is 1. For example, let **x1** be the vector shown below.

1 1 1 0 1 0 1 1 The reduction AND (& x1) operation is equivalent to the following operation: 1 & 1 & 1 & 0 & 1 & 0 & 1 & 1 which returns a result of **1**'b**0**.
For the **reduction NAND** operator, if any bit in the operand is 0, then the result is 1; otherwise, the result is 0. For a vector x1 , the reduction NAND (~& x1) is the inverse of the reduction AND operator.
For the **reduction OR** operator, if any bit in the operand is 1, then the result is 1; otherwise, the result is 0. For example, let x1 be the vector shown below. 1 1 1 0 1 0 1 1 The reduction OR (| x1) operation is equivalent to the following operation: 1 | 1 | 1 | 0 | 1 | 0 | 1 | 1 which returns a result of **1**'b**1.**
For the **reduction NOR** operator, if any bit in the operand is 1, then the result is 0; otherwise, the result is 1. For a vector x1 , the reduction NOR (~ | x1) is the inverse of the reduction OR operator.
For the exclusive-OR operator, if there are an even number of 1s in the operand, then the result is 0; otherwise, the result is 1. For example, let x1 be the vector shown below. 1 1 1 0 1 0 1 1 The reduction exclusive-OR (^ x1) operation is equivalent to the following operation: 1 ^ 1 ^ 1 ^ 0 ^ 1 ^ 0 ^ 1 ^ 1 which returns a result of **1**'b**0**. The reduction exclusive-OR operator can be used as an even parity generator.
For the **exclusive-NOR operator**, if there are an odd number of 1s in the operand, then the result is 0; otherwise, the result is 1. For a vector x1 , the reduction exclusiveNOR (^ ~ x1) is the inverse of the reduction exclusive-OR operator. The reduction exclusive-NOR operator can be used as an odd parity generator.

module my_design; **reg [7:0] x1;** **reg [7:0] red_and;** **initial** **begin** **x1=4'b0001;** **red_and=&x1;** **$display("red_and=%b",red_and);** **end** **endmodule**	**# run 1000ns** **red_and=00000000**

8-bit representation: input ,output

module my_design;	**# run 1000ns**

```
reg [7:0] a;
reg [7:0] red_and,red_nand,red_or,red_nor,red_xor,red_xnor;

   initial
   begin
   a=4'b0001;
   red_and=&a;       //reduction AND
   red_nand=~&a;       //reduction NAND
   red_or=|a;          //reduction OR
   red_nor = ~|a;      //reduction NOR
   red_xor     =     ^a;       //reduction exclusive-OR
   red_xnor = ^~a;         //reduction exclusive-NOR
   $display("red_and=%b",red_and);
   $display("red_nand=%b",red_nand);
   $display("red_or=%b",red_or);
   $display("red_nor=%b",red_nor);
   $display("red_xor=%b",red_xor);
   $display("red_xnor=%b",red_xnor);

   end
endmodule
```

```
red_and=00000000
red_nand=00000001
red_or=00000001
red_nor=00000000
red_xor=00000001
red_xnor=00000000
```

Register: input 4-bit,output 1 bit(default)

```
module my_design;

reg [3:0] a;
reg red_and,red_nand,red_or,red_nor,red_xor,red_xnor;

   initial
   begin
   a=4'b0001;
   red_and=&a;       //reduction AND
   red_nand=~&a;       //reduction NAND
   red_or=|a;          //reduction OR
   red_nor = ~|a;      //reduction NOR
   red_xor     =     ^a;       //reduction exclusive-OR
```

```
# run 1000ns
a=0001
red_and=0
red_nand=1
red_or=1
red_nor=0
red_xor=1
red_xnor=0
```

```
   red_xnor = ^~a;        //reduction exclusive-NOR
   $display("red_and=%b",red_and);
   $display("red_nand=%b",red_nand);
   $display("red_or=%b",red_or);
   $display("red_nor=%b",red_nor);
   $display("red_xor=%b",red_xor);
   $display("red_xnor=%b",red_xnor);
   End

endmodule
```

Shift:

The **shift operators** shift a single vector operand **left** or **right** a specified number of **bit** positions. These are logical shift operations, not algebraic; that is, as bits are shifted left or right, zeroes fill in the vacated bit positions.

The bits shifted out of the operand are lost; they do not rotate to the high-order or low-order bit positions of the shifted operand. If the shift amount evaluates to **x** or **z**, then the result of the operation is **x**. There are two shift operators, as shown below. The value in parentheses is the number of bits that the operand is shifted.

<< (Left-shift amount)

>> (Right-shift amount)

1.When an **operand** is **shifted left**, this is equivalent to a **multiply-by-two** operation for each bit position shifted.

2.When an operand is **shifted right**, this is equivalent to a **divide-by-two** operation for each bit position shifted.

The shift operators are useful to model the sequential add-shift multiplication algorithm **and the sequential** shift-subtract division algorithm**.**

a_reg = 00000010, b_reg = 00001000, //shift a_reg left 3

rslt_a = 00010000, rslt_b = 00000010 //shift b_reg right 2

```
module my_design;

reg [7:0]  a_reg,b_reg;
reg  [7:0] rslt_a,rslt_b;

   initial
   begin
   a_reg = 00000010;b_reg = 00001000;
   rslt_a=a_reg<<3; //multiply by 8  (2^3=8)
```

```
# run 1000ns
a_reg=00001010 a_reg= 10
b_reg=11101000 b_reg=232
rslt_a=01010000 rslt_a= 80
rslt_b=00111010  rslt_b= 58
```

```
    rslt_b=b_reg>>2; //divide by 4  (2^2=4)
    $display("a_reg=%b a_reg=%d",a_reg,a_reg);
    $display("b_reg=%b b_reg=%d ",b_reg,b_reg);
    $display(" rslt_a=%b rslt_a=%d", rslt_a,rslt_a);
    $display(" rslt_b=%b  rslt_b=%d", rslt_b,rslt_b);
    end
endmodule
```

Input 8-bit: a_reg ,b_reg

Output 8-bit: rslt_a,rslt_b

```
module my_design;

reg [7:0]  a_reg,b_reg;
reg  [7:0] rslt_a,rslt_b;

    initial
    begin

    a_reg = 00000010;b_reg = 00001000;

    $display("a  reg=%b a  reg=%d",a  reg,a  reg);
```

```
# run 1000ns
a_reg=00001010  a_reg= 10
 rslt_a=00010100 rslt_a= 20
 rslt_a=00101000 rslt_a= 40
 rslt_a=01010000 rslt_a= 80
b_reg=11101000 b_reg=232
 rslt_b=01110100  rslt_b=116
 rslt_b=00111010  rslt_b= 58
 rslt_b=00011101  rslt_b= 29
```

```
    rslt_a=a_reg<<1;    //multiply by 2  (2^3=8)
    $display(" rslt_a=%b rslt_a=%d", rslt_a,rslt_a);

    rslt_a=a_reg<<2; //multiply by 4  (2^2=4)
    $display(" rslt_a=%b rslt_a=%d", rslt_a,rslt_a);

    rslt_a=a_reg<<3; //multiply by 8  (2^3=8)
    $display(" rslt_a=%b rslt_a=%d", rslt_a,rslt_a);



    $display("b_reg=%b b_reg=%d ",b_reg,b_reg);
    rslt_b=b_reg>>1; //divide by 2  (2^1=4)
    $display(" rslt_b=%b  rslt_b=%d", rslt_b,rslt_b);

    rslt_b=b_reg>>2; //divide by 4  (2^2=4)
    $display(" rslt_b=%b  rslt_b=%d", rslt_b,rslt_b);

    rslt_b=b_reg>>3; //divide by 8  (2^3=4)
    $display(" rslt_b=%b  rslt_b=%d", rslt_b,rslt_b);

    end
endmodule
```

Conditional:

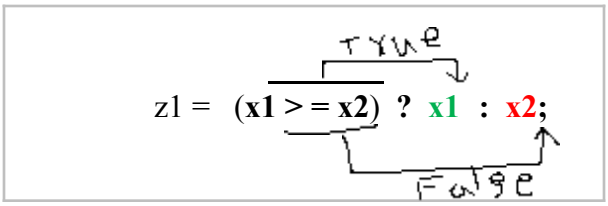
The conditional operator **(? :)** has three operands, as shown in the syntax below. The conditional_expression is evaluated. If the result is true (1), then the true_expression is evaluated; if the result is false (0), then the false_expression is evaluated.

conditional_expression ? true_expression **:** false_expression**;**

The **conditional operator** can be used when one of two expressions is to be selected.

For example, in the statement below, if **x1** is greater than or equal to x2 , then z1 is assigned the value of x3 ; if x1 is less than x2 , then z1 is assigned the value of x4 .

z1 = **(x1 > = x2) ? x1 : x2 ;**



The conditional operator is ideally suited to model a **2:1** multiplexer. Conditional operators can be nested; that is, each true_expression and false_expression can be a conditional operation.

module my_design; **reg [7:0] x1,x2;** **reg [7:0] z1;** **initial** **begin** **x1=5;x2=10;** **z1=(x1>x2)?x1:x2;** **$display("x1=%0d x2=%0d z1=%0d", x1,x2,z1);** **end** **endmodule**	**# run 1000ns** **x1=5 x2=10 z1=10**

module my_design; **reg [7:0] x1,x2;** **reg [7:0] z1;** **initial** **begin** **x1=10;x2=5;** **z1=(x1>x2)?x1:x2;** **$display("x1=%0d x2=%0d z1=%0d", x1,x2,z1);** **end** **endmodule**	**# run 1000ns** **x1=10 x2=5 z1=10**

conditional_expression ? (cond_expr1 ? true_expr1 : false_expr1) : (cond_expr2 ? true_expr2 : false_expr2);

z1=(x1>x2)?((x1>x3)?x1:x3):x2;

Find biggest number:

module my_design;	**# run 1000ns**

reg [7:0] x1,x2,x3;
reg [7:0] z1;
 	
initial
begin
 	
x1=10;x2=15;x3=20;
z1=(x1>x2)?((x1>x3)?x1:x3):x2;
 	
$display("x1=%0d x2=%0d z1=%0d", x1,x2,z1);
 	
end
endmodule
	x1=10 x2=15 z1=15

module my_design;
 	
reg [7:0] x1,x2,x3;
reg [7:0] z1;
 	
initial
begin
 	
x1=10;x2=15;x3=20;
z1=(x1>x2)?((x1>x3)?x1:x3):((x2>x3)?x2 :x3);
 	
$display("x1=%0d x2=%0d z1=%0d", x1,x2,z1);
 	
end
endmodule
	# run 1000ns
	x1=10 x2=15 z1=15

Concatenation:

The **concatenation operator** ({ }) forms a single operand from **two** or **more** operands by joining the different operands in sequence separated by commas. The operands to be appended are contained within braces. The size of the **operands** must be known before concatenation takes place.

The examples below show the concatenation of scalars and vectors of different sizes. Outputs z1 , z2 , z3 , and z4 are ten bits in length.

z1 , z2 , z3 , and z4 are 10 bits in length . a = 11, b = 001, c = 1100, d = 1

```
z1 = 0000_11_1100        //z1 = {a, c}
 z2 = 00000_001_11       //z2 = {b, a}
z3 = 0_1100_001_11       //z3 = {c, b, a}
z4 = 11  001  1100  1    //z4 = {a, b, c, d}
```

```
module my_design;

reg [0:1] a;
reg [0:2] b;
reg [0:3] c;
reg d;

reg [9:0]  z1,z2,z3,z4;
   initial
   begin
   a = 11;b=001;c=1100;d=1;
   z1={a,c};
   z2={b,c};
   z3={c,b,a};
   z4={a,b,c,d};

   $display("z1=%b",z1);
   $display("z2=%b",z2);
   $display("z3=%b",z3);
   $display("z4=%b",z4);


   end
endmodule
```

```
# run 1000ns
z1=0000111100
z2=0000011100
z3=0110000111
z4=1100111001
```

a = 11;b=001;c=1100;d=1;

```
module my_design;

reg [1:0] a;
reg [2:0] b;
reg [3:0] c;
reg d;
```

```
reg [9:0]  z1,z2,z3,z4;


   initial
   begin
   a = 11;b=001;c=1100;d=1;
   z1={a,c};
   z2={b,c};
   z3={c,b,a};
   z4={a,b,c,d};

   $display("z1=%b",z1);
   $display("z2=%b",z2);
   $display("z3=%b",z3);
   $display("z4=%b",z4);


   end
endmodule
```

Replication:

Replication is a means of performing **repetitive concatenation**.

Replication specifies the number of **times to duplicate** the expressions within the innermost braces. The syntax is:

{**number_ of_ repetitions** {expression_1, expression_2, . . . , expression_n}};

```
a = 11, b = 010, c = 0011,

z1 = 11_0011_11_0011,     //z1 = {2{a, c}}
z2 = 010  0011  0111  010  0011  0111    //z2 = {2{b, c, 4'b0111}}
```

```
module my_design;

reg [1:0] a;
reg [2:0] b;
reg [3:0] c;

reg [31:0]  z1,z2;

   initial
   begin
   a=2'b11;b=3'b010;c=4'b0011;
```

```
# run 1000ns
z1=11001110011
z2=10001101110100011 0111
```

```
   z1={2{a,c}};
   z2={2{b,c,4'b0111}};

   $display("z1=%0b",z1);
   $display("z2=%0b",z2);



   end
endmodule
```

```
module my_design;

reg [1:0] a;
reg [2:0] b;
reg [3:0] c;

reg [31:0]  z1,z2;

   initial
   begin
   a=2'b11;b=3'b001;c=4'b1100;
   z1={2{a,c}};
   z2={2{b,c,4'b0111}};

   $display("z1=%0b",z1);
   $display("z2=%0b",z2);



   end
endmodule
```

Display 1 to 15 binary and decimal numbers

```
module my_design;
integer i=1;

initial
  for(i=1;i<=15;i=i+1)
    begin
    $display("%0d=%0b",i,i);
    End
```

```
# run 1000ns
1=1
2=10
3=11
4=100
5=101
6=110
7=111
8=1000
9=1001
10=1010
```

endmodule	11=1011 12=1100 13=1101 14=1110 15=1111

Expressions:

Expressions consist of operands and operators, which are the basis of Verilog HDL. The result of a right-hand side expression can be assigned to a left-hand side net vari able or register variable using the keyword assign.

assign z1 = x1 & x2 & x3;

assign z1 = x1 | x2 | x3;

assign cout = (a & cin) | (b & cin) | (a & b);

Modules and Ports:

A **module** is the basic unit of design in **Verilog**. It describes the functional operation of some logical entity and can be a stand-alone module or a collection of modules that are instantiated into a structural module. Instantiation means to use one or more lower level modules in the construction of a higher-level structural module. A **module** can be a logic gate, an adder, a multiplexer, a counter, or some other logical function.

A module consists of declarative text which specifies the function of the module using Verilog constructs; that is, a Verilog module is a **software representation** of the **physical hardware structure** and **behavior.**

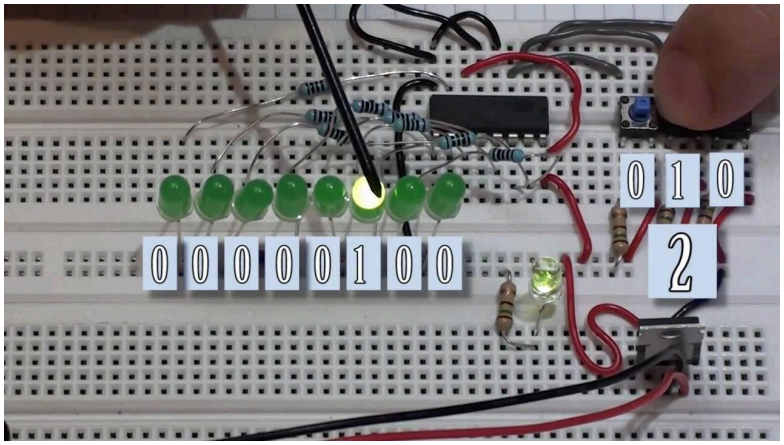
The declaration of a module is indicated by the keyword **module** and is always terminated by the keyword **endmodule.**

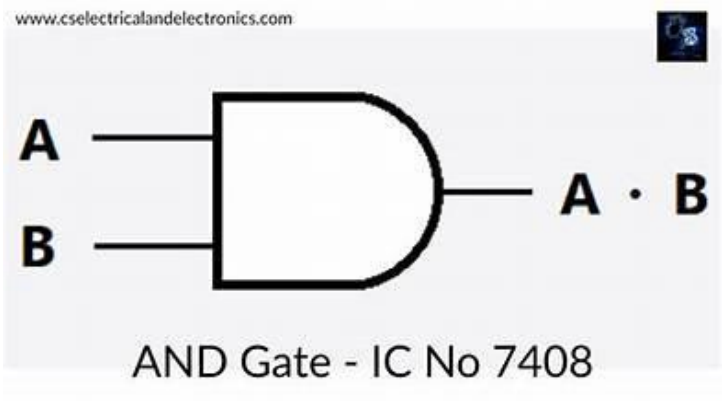
Verilog has predefined logical elements called **primitives**. These built-in primi tives are structural elements that can be instantiated into a larger design to form a more complex structure. Examples are: and, or, xor, and not.

```
module module_name(port list);
  declarations
      reg, wire, parameter,
      input, output, . . .
```

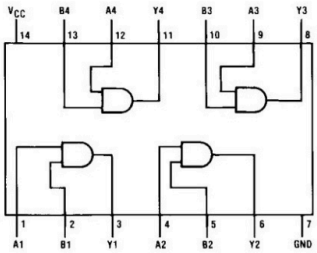
```
    . . .
     <module internals>
       statements
       initial, always, module instantiation, . . .
       . . .
endmodule
```

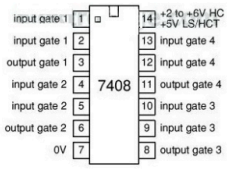
Verilog module structure

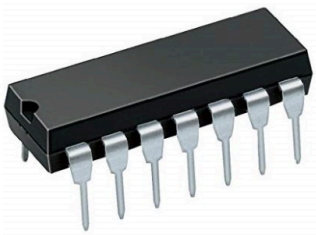




AND GATE IC 7408







Level of abstraction:

1.Switch level : Implementaions are done using switches.

2.Gate-level modeling:logic gates.

3.Data flow level modeling: Designing is done interms of equation**.**

4.Behavioural level modeling: Based on behaviour of circuits.

Gate-level modeling	Data flow level modeling	Behavioural level modeling

Gate-level modeling:

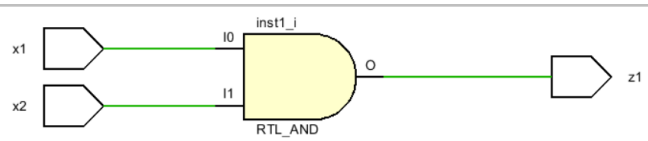
- Model is implemeted interms of logic gates.
- It is lowest level of abstration.
- Basic gates are available in **predefined primitives**. Gate primitives are predefined modules in Verilog, which are ready to use.

Syntax:

Primitivegate_name instatnce_name(output,input1,input2,**input..n**);

1.

```
module and_gate(z1,x1,x2);

input x1,x2;
output z1;

and gate(z1,x1,x2);

endmodule
```



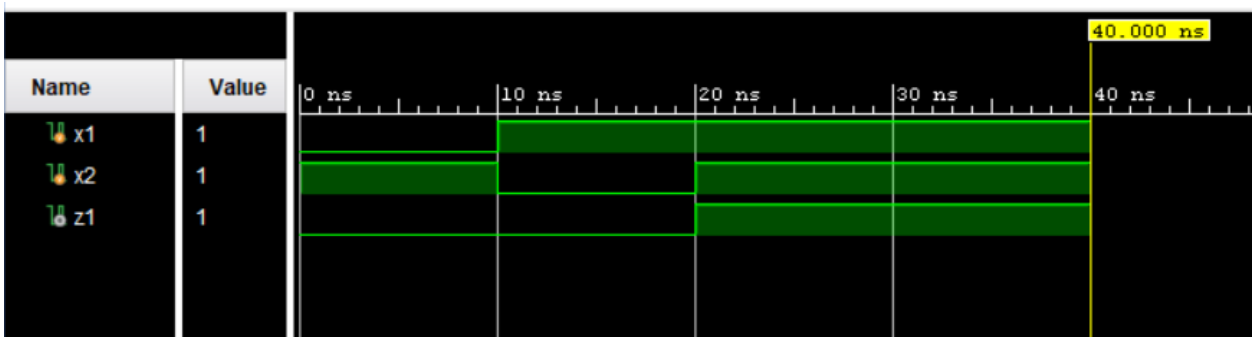
Verilog module for an AND gate

2.

```
//and_gate test bench
module and_gate_tb;
reg x1, x2;
wire z1;
```

```
//apply input vectors
initial
begin
x1 = 1'b0;x2 = 1'b0;
#0 $display("x1 = %b, x2 = %b, z1 = %b", x1, x2, z1);
x1 = 1'b0;x2 = 1'b1;
#10 $display("x1 = %b, x2 = %b, z1 = %b", x1, x2, z1);
x1 = 1'b1;x2 = 1'b0;
#10 $display("x1 = %b, x2 = %b, z1 = %b", x1, x2, z1);
x1 = 1'b1;x2 = 1'b1;
#10 $display("x1 = %b, x2 = %b, z1 = %b", x1, x2, z1);
#10 $stop;
end
//instantiate the module into the test bench
and_gate UUT(
            .x1(x1),
            .x2(x2),
            .z1(z1)
            );
endmodule
```

Test bench for the AND gate



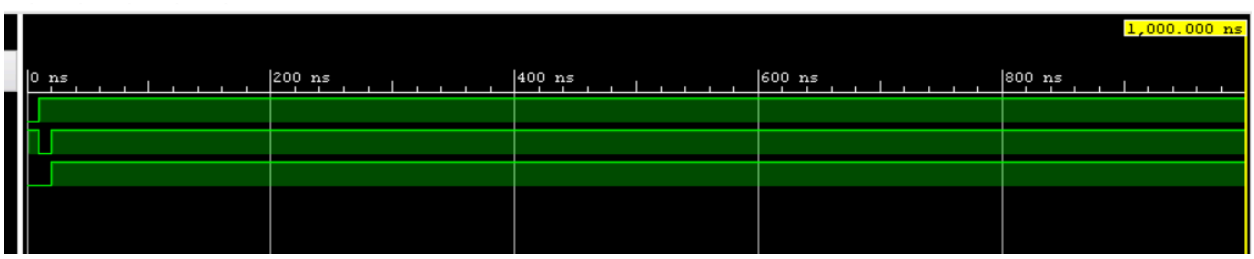
Output waveform

```
# run 1000ns
x1 = 0, x2 = 0, z1 = 0
x1 = 0, x2 = 1, z1 = 0
x1 = 1, x2 = 0, z1 = 0
x1 = 1, x2 = 1, z1 = 1
```

Binary outputs for the test bench

With out $stop:

```
//and_gate test bench
module and_gate_tb;
reg x1, x2;
wire z1;

//apply input vectors
initial
begin
x1 = 1'b0;x2 = 1'b0;
#0 $display("x1 = %b, x2 = %b, z1 = %b", x1, x2, z1);
x1 = 1'b0;x2 = 1'b1;
#10 $display("x1 = %b, x2 = %b, z1 = %b", x1, x2, z1);
x1 = 1'b1;x2 = 1'b0;
#10 $display("x1 = %b, x2 = %b, z1 = %b", x1, x2, z1);
x1 = 1'b1;x2 = 1'b1;
#10 $display("x1 = %b, x2 = %b, z1 = %b", x1, x2, z1);

end
//instantiate the module into the test bench
and_gate UUT(
        .x1(x1),
        .x2(x2),
        .z1(z1)
        );
endmodule
```



3

```
//and_gate test bench
module and_gate_tb;
reg x1, x2;
wire z1;
```

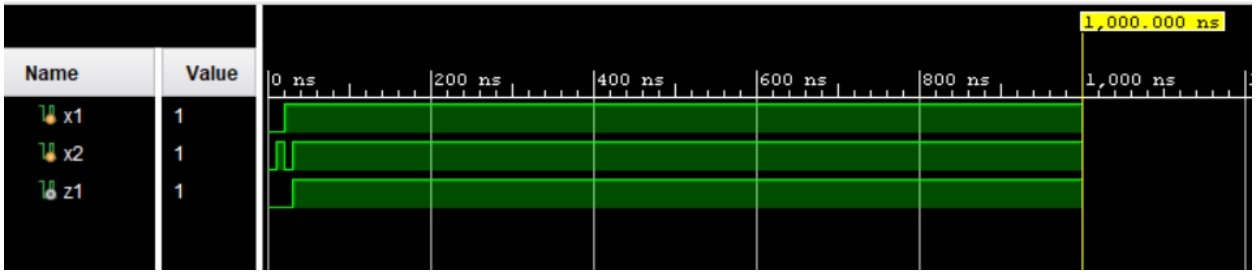
```
//display variables
initial
$monitor("x1 = %b, x2 = %b, z1 = %b", x1, x2, z1);

//apply input vectors
initial
begin
#0  x1 = 1'b0;x2 = 1'b0;
#10 x1 = 1'b0;x2 = 1'b1;
#10 x1 = 1'b1;x2 = 1'b0;
#10 x1 = 1'b1;x2 = 1'b1;
end
//instantiate the module into the test bench
and_gate UUT(
            .x1(x1),
            .x2(x2),
            .z1(z1)
            );
endmodule
```

3

$monitor:

```
//and_gate test bench
module and_gate_tb;
reg x1, x2;
wire z1;

//display variables
initial
$monitor("x1 = %b, x2 = %b, z1 = %b", x1, x2, z1);

//apply input vectors
initial
begin
#0  x1 = 1'b0;x2 = 1'b0;
#10 x1 = 1'b0;x2 = 1'b1;
#10 x1 = 1'b1;x2 = 1'b0;
#10 x1 = 1'b1;x2 = 1'b1;

end
//instantiate the module into the test bench
and_gate UUT(
```

```
                .x1(x1),
                .x2(x2),
                .z1(z1)
                );
endmodule
```




```
# run 1000ns
                    x1 = 0, x2 = 0, z1 = 0
                    x1 = 0, x2 = 1, z1 = 0
                    x1 = 1, x2 = 0, z1 = 0
                    x1 = 1, x2 = 1, z1 = 1
```

3.1

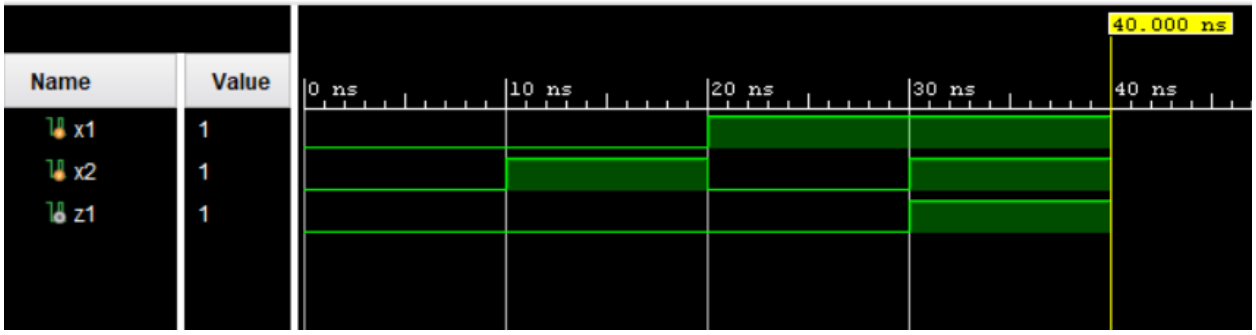
```
//and_gate test bench
module and_gate_tb;
reg x1, x2;
wire z1;

//display variables
initial
$monitor("x1 = %b, x2 = %b, z1 = %b", x1, x2, z1);

//apply input vectors
initial
begin
#0  x1 = 1'b0;x2 = 1'b0;
#10 x1 = 1'b0;x2 = 1'b1;
#10 x1 = 1'b1;x2 = 1'b0;
#10 x1 = 1'b1;x2 = 1'b1;
#10 $stop;
end
//instantiate the module into the test bench
and_gate UUT(
            .x1(x1),
            .x2(x2),
```

```
            .z1(z1)
            );
endmodule
```

3.2 $time:

Time limit: #10

```
//and_gate test bench
module and_gate_tb;
reg x1, x2;
wire z1;

//display variables
initial
$monitor($time,"ns x1 = %b, x2 = %b, z1 = %b", x1, x2, z1);

//apply input vectors
initial
begin
#0  x1 = 1'b0;x2 = 1'b0;
#10 x1 = 1'b0;x2 = 1'b1;
#10 x1 = 1'b1;x2 = 1'b0;
#10 x1 = 1'b1;x2 = 1'b1;
#10 $stop;
end
//instantiate the module into the test bench
and_gate UUT(
            .x1(x1),
            .x2(x2),
            .z1(z1)
            );
endmodule
```

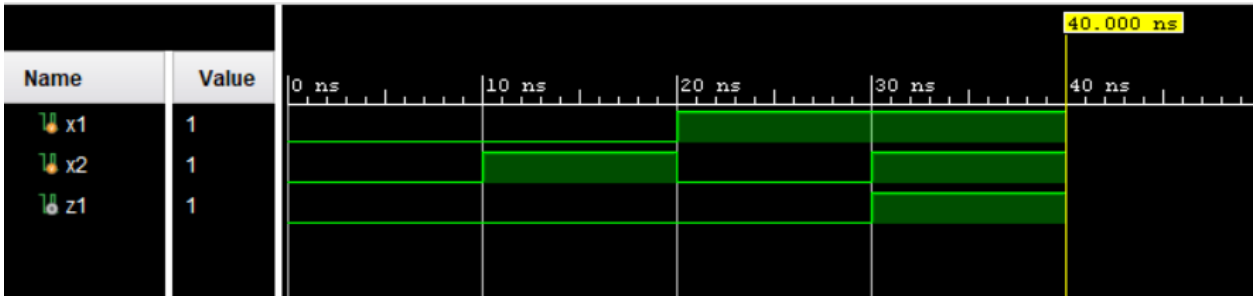



```
# run 1000ns
                    0ns x1 = 0, x2 = 0, z1 = 0
                   10ns x1 = 0, x2 = 1, z1 = 0
                   20ns x1 = 1, x2 = 0, z1 = 0
                   30ns x1 = 1, x2 = 1, z1 = 1
```

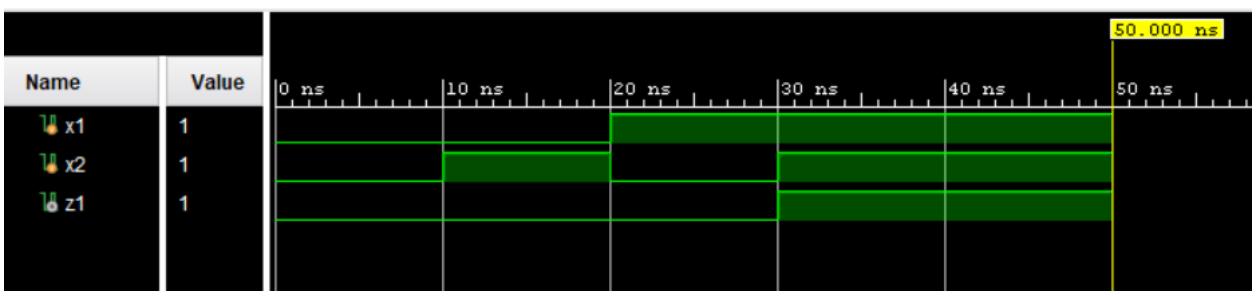
3.5

With LOOP:

```
//and_gate test bench
module and_gate_tb;
reg x1, x2;
wire z1;

// vectors inputs applying
initial
begin:loop_starts
 reg [3:0] i;
 for(i=0;i<4;i=i+1)
   begin
   {x1,x2}=i[3:0];
   #10 $display("%b %b",{x1,x2},z1);
   end
#0 $stop;
end
// instantiate the module into test bench
and_gate UUT(
            .x1(x1),
            .x2(x2),
            .z1(z1)
            );
endmodule
```




```
//and_gate test bench
module and_gate_tb;
reg x1, x2;
wire z1;

// vectors inputs applying
initial
begin:loop_starts
 reg [3:0] i;
 for(i=0;i<4;i=i+1)
   begin
   {x1,x2}=i[3:0];
   #10 $display("%b %b",{x1,x2},z1);
   end
#10 $stop;
end
// the test bench instantiate the module
and_gate UUT (
            .x1(x1),
            .x2(x2),
            .z1(z1)
            );
endmodule
```




```
//and_gate test bench
module and_gate_tb;
reg x1, x2;
```

```
wire z1;

//apply input vectors
initial
begin:loop_starts
 reg [3:0] i;
 for(i=0;i<4;i=i+1)
   begin
   {x1,x2}=i[3:0];
   #10 $display("x1x2=%b %b",{x1,x2},z1);
   end
#10 $stop;
end
//instantiate the module into the test bench
and_gate UUT (
            .x1(x1),
            .x2(x2),
            .z1(z1)
            );
endmodule
```

```
# run 1000ns
x1x2=00 0
x1x2=01 0
x1x2=10 0
x1x2=11 1
```

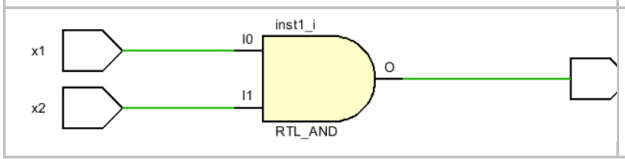
With out loop: #10 $stop

With loop :#0 $stop

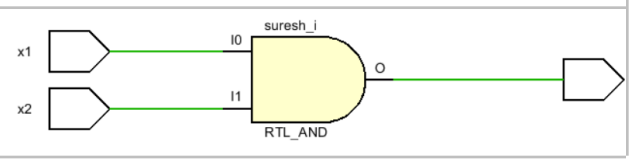
Verilog code for AND gate using gate-level modeling:

Example 1: And gate

```
module and_gate(z1,x1,x2);

input x1,x2;
output z1;

and UUT(z1,x1,x2);

endmodule
```




```
module and_gate(z1,x1,x2);

input x1,x2;
output z1;

and suresh(z1,x1,x2);

endmodule
```



Verilog module for an AND gate

Designing a Test Bench for Simulation:

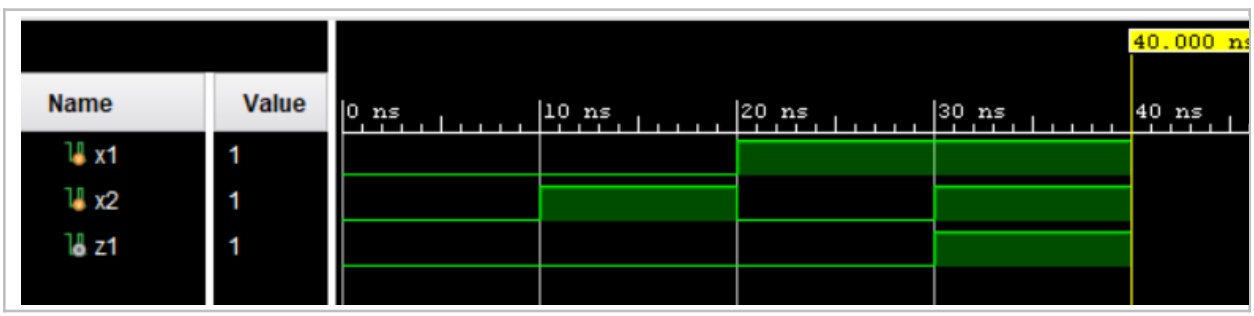
```
//and_gate test bench
module and_gate_tb;
reg x1, x2;
wire z1;

//display variables
initial
$monitor($time,"ns x1 = %b, x2 = %b, z1 = %b", x1, x2, z1);

//apply input vectors
initial
begin
#0  x1 = 1'b0;x2 = 1'b0;
#10 x1 = 1'b0;x2 = 1'b1;
#10 x1 = 1'b1;x2 = 1'b0;
#10 x1 = 1'b1;x2 = 1'b1;
```

```
#10 $stop;
end
//instantiate the module into the test bench
and_gate inst1 (
            .x1(x1),
            .x2(x2),
            .z1(z1)
            );
endmodule
```

Test bench for the AND gate

```
# run 1000ns
                    0ns x1 = 0, x2 = 0, z1 = 0
                   10ns x1 = 0, x2 = 1, z1 = 0
                   20ns x1 = 1, x2 = 0, z1 = 0
                   30ns x1 = 1, x2 = 1, z1 = 1
```

Binary outputs for the test bench

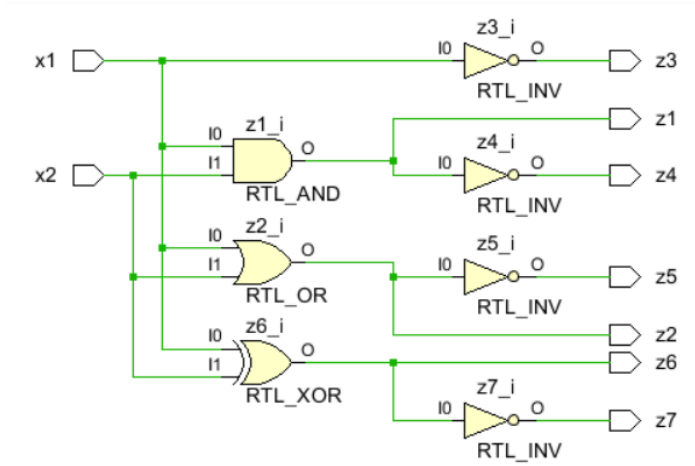


Output waveform

Predefined premitive gates:

All logic gates can be implemeted using these gates.

1.basic gates




```
module basic_gates(x1,x2,z1,z2,z3,z4,z5,z6,z7);

input x1,x2;
output z1,z2,z3,z4,z4,z5,z6,z7;

and (z1,x1,x2);
or (z2,x1,x2);
not (z3,x1);
nand  (z4,x1,x2);
nor (z5,x1,x2);
xor (z6,x1,x2);
xnor (z7,x1,x2);

endmodule
```

Verilog module for basic gates

Designing a Test Bench for Simulation:

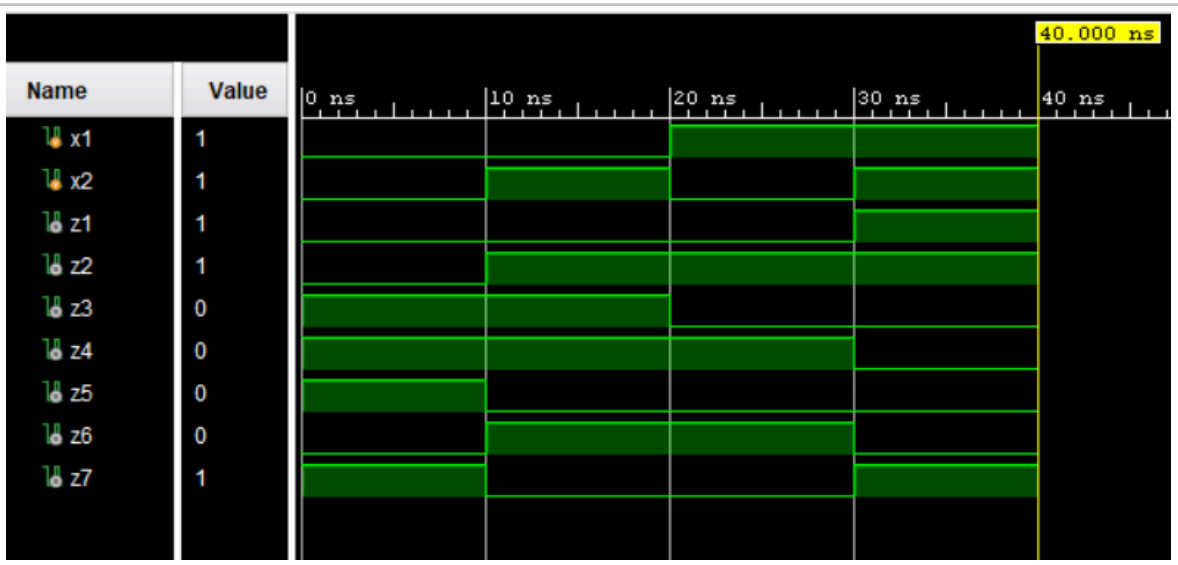
```
//basic_gates test bench
module basic_gates_tb;

reg x1, x2;
wire z1,z2,z3,z4,z5,z6,z7;

//display variables
initial
$monitor($time,"ns x1 = %b, x2 = %b, and gate:z1 = %b, or
gate:z2=%b, not gate:z3=%b, nand gate:z4=%b, nor gate:z5=%b,
ex-or:z6=%b,ex-nor:z7=%b", x1, x2, z1,z2,z3,z4,z5,z6,z7);

//apply input vectors
initial
```

```
begin
  #0  x1 = 1'b0;x2 = 1'b0;
  #10 x1 = 1'b0;x2 = 1'b1;
  #10 x1 = 1'b1;x2 = 1'b0;
  #10 x1 = 1'b1;x2 = 1'b1;
  #10 $stop;
end

//instantiate the module into the test bench
basic_gates uut(
            .x1(x1),
            .x2(x2),
            .z1(z1),
            .z2(z2),
            .z3(z3),
            .z4(z4),
            .z5(z5),
            .z6(z6),
            .z7(z7)
            );
endmodule
```

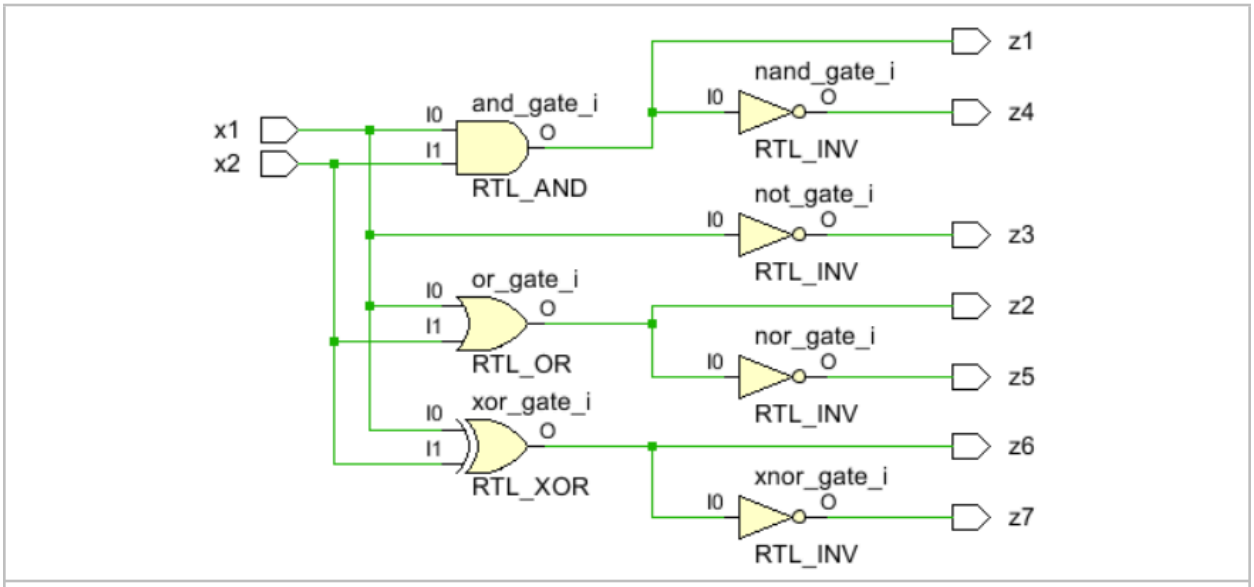
Test bench for basic gates

```
 0ns x1 = 0, x2 = 0, and gate:z1 = 0, or gate:z2=0, not gate:z3=1, nand gate:z4=1, nor gate:z5=1, ex-or:z6=0,ex-nor:z7
10ns x1 = 0, x2 = 1, and gate:z1 = 0, or gate:z2=1, not gate:z3=1, nand gate:z4=1, nor gate:z5=0, ex-or:z6=1,ex-nor:z7
20ns x1 = 1, x2 = 0, and gate:z1 = 0, or gate:z2=1, not gate:z3=0, nand gate:z4=1, nor gate:z5=0, ex-or:z6=1,ex-nor:z7
30ns x1 = 1, x2 = 1, and gate:z1 = 1, or gate:z2=1, not gate:z3=0, nand gate:z4=0, nor gate:z5=0, ex-or:z6=0,ex-nor:z7
```

Binary outputs for the test bench



Output waveform




```
module basic_gates(x1,x2,z1,z2,z3,z4,z5,z6,z7);

input x1,x2;
output z1,z2,z3,z4,z4,z5,z6,z7;

and and_gate(z1,x1,x2);
or or_gate(z2,x1,x2);
not not_gate(z3,x1);
nand  nand_gate(z4,x1,x2);
nor nor_gate(z5,x1,x2);
xor xor_gate(z6,x1,x2);
xnor xnor_gate(z7,x1,x2);

endmodule
```

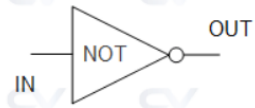
2.buf if/not if

Buf/Not Gates:

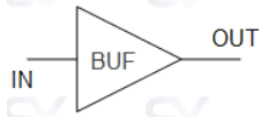
These gates have only **one scalar** input and **one or more outputs**.

buf stands for a **buffer** and simply transfer the value from **input** to the **output** without any change in polarity.

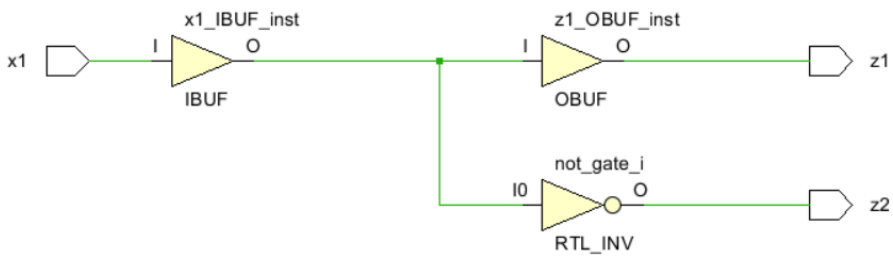
not stands for an **inverter** which inverts the polarity of the signal at its input. So a **0** at its input will yield a **1** and vice versa.



IN	OUT
0	1
1	0
X/Z	X



IN	OUT
0	0
1	1
X/Z	X




```
module gates(x1,z1,z2);
input x1;
output z1,z2;
buf buf_gate(z1,x1);
not not_gate(z2,x1);
endmodule
```

```
module gates_tb;
reg x1;
wire z1,z2;
initial
$monitor($time,"ns x1=%b,z1=%b,z2=%b",x1,z1,z2);
initial
begin
#0    x1=1'b0;
#10    x1=1'b1;
#10    x1=1'b0;
#10    x1=1'b1;
#10 $stop;
end
endmodule
```

data-flow modeling:

- Register transfer level.
- Module is designed by specific data flow.

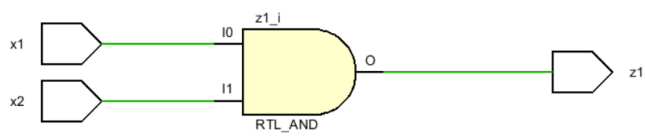
- Signals are designed by data manupulating equations.
- Design is implemeted continuous assignments.

'asssign' keyword is used.

```
assign z1 = x1 & x2;      //and
assign z2 = x1 | x2;     //or
assign z3 = !x1;        //not
assign z4 = x1 ^ x2;  //exclusive-OR
assign z5 = x1 ~^ x2;//exclusive-NOR
```

Verilog code for AND gate using data-flow modeling:

Example 1: And gate




```
//dataflow and gate_gate with two inputs
module and_gate(x1, x2, z1);
   input x1, x2;
   output z1;
   wire x1, x2;
   wire z1;
   assign z1 = x1 & x2;
endmodule
```

Verilog module for an AND gate with two inputs

Designing a Test Bench for Simulation:

```
//and_gate test bench
module and_gate_tb;
reg x1, x2;
wire z1;

//display variables
initial
$monitor($time,"ns x1 = %b, x2 = %b, z1 = %b", x1, x2, z1);

//apply input vectors
initial
begin
```

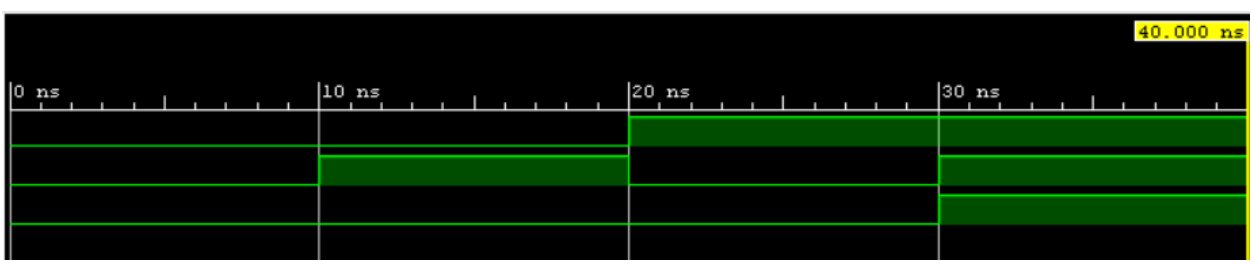
```
#0  x1 = 1'b0;x2 = 1'b0;
#10 x1 = 1'b0;x2 = 1'b1;
#10 x1 = 1'b1;x2 = 1'b0;
#10 x1 = 1'b1;x2 = 1'b1;
#10 $stop;
end
//instantiate the module into the test bench
and_gate inst1 (
            .x1(x1),
            .x2(x2),
            .z1(z1)
            );
endmodule
```

Test bench for the AND gate

Result:

```
# run 1000ns
x1 = 0, x2 = 0, z1 = 0
x1 = 0, x2 = 1, z1 = 0
x1 = 1, x2 = 0, z1 = 0
x1 = 1, x2 = 1, z1 = 1
```

Binary outputs for the test bench



Output waveform

Example2:

Logic primitives: AND (&),

OR (|),

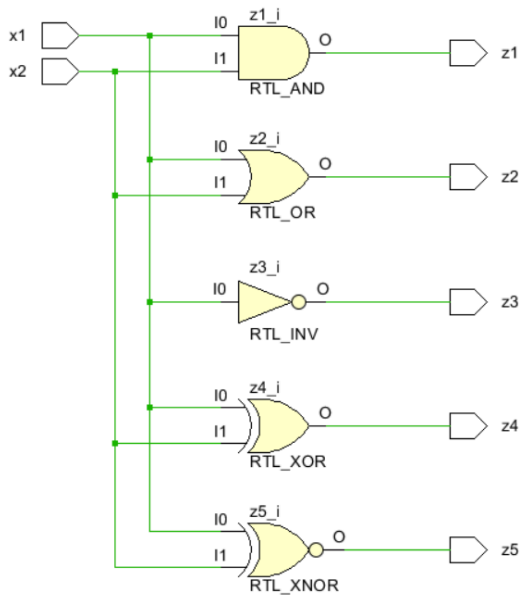
negation (~),

exclusive-OR (^), and

exclusive-NOR (^ ~ or ~ ^).

The **outputs** are declared as **wire**; the **inputs** can be declared as either **wire** or **reg**

Symbol:




```
//dataflow modelling ,logic gates
module gates(x1, x2, z1,z2,z3,z4,z5);

   input x1, x2;
   output z1, z2, z3, z4, z5;

   wire x1, x2;
   wire z1, z2, z3, z4, z5;

   assign z1 = x1 & x2;      //and
   assign z2 = x1 | x2;     //or
   assign z3 = !x1;        //not
   assign z4 = x1 ^ x2;   //exclusive-OR
   assign z5 = x1 ~^ x2;//exclusive-NOR

endmodule
```

Verilog module for an logic gates

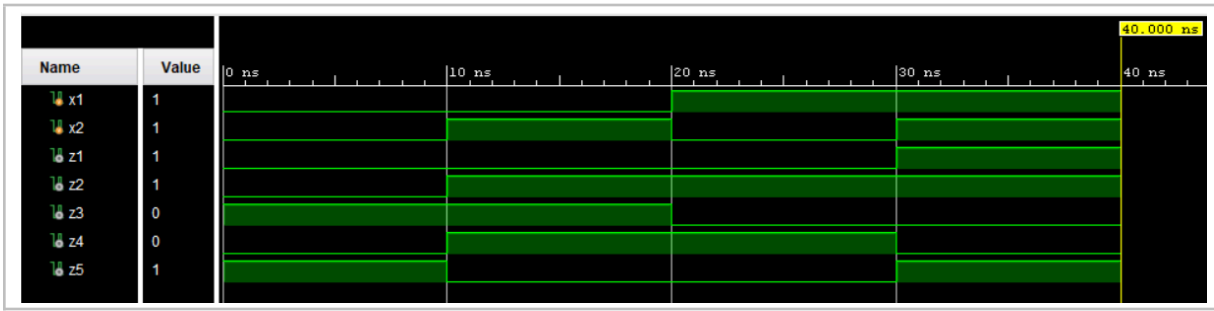
```
//and_gate test bench
module gates_tb;

reg x1, x2;
```

```
wire z1,z2,z3,z4,z5;

//display variables
initial
$monitor($time,"ns  x1  =  %b,  x2  =  %b,  z1  =  %b,  z2=%b,
z3=%b,z4=%b, z5=%b", x1, x2, z1,z2,z3,z4,z5);

//apply input vectors
initial
begin
  #0  x1 = 1'b0;x2 = 1'b0;
  #10 x1 = 1'b0;x2 = 1'b1;
  #10 x1 = 1'b1;x2 = 1'b0;
  #10 x1 = 1'b1;x2 = 1'b1;
  #10 $stop;
End

//instantiate the module into the test bench
gates uut(
            .x1(x1),
            .x2(x2),
            .z1(z1),
            .z2(z2),
            .z3(z3),
            .z4(z4),
            .z5(z5)
            );
endmodule
```

Test bench for the gates

```
# run 1000ns
                    0ns x1 = 0, x2 = 0, z1 = 0, z2=0, z3=1,z4=0, z5=1
                   10ns x1 = 0, x2 = 1, z1 = 0, z2=1, z3=1,z4=1, z5=0
                   20ns x1 = 1, x2 = 0, z1 = 0, z2=1, z3=0,z4=1, z5=0
                   30ns x1 = 1, x2 = 1, z1 = 1, z2=1, z3=0,z4=0, z5=1
```

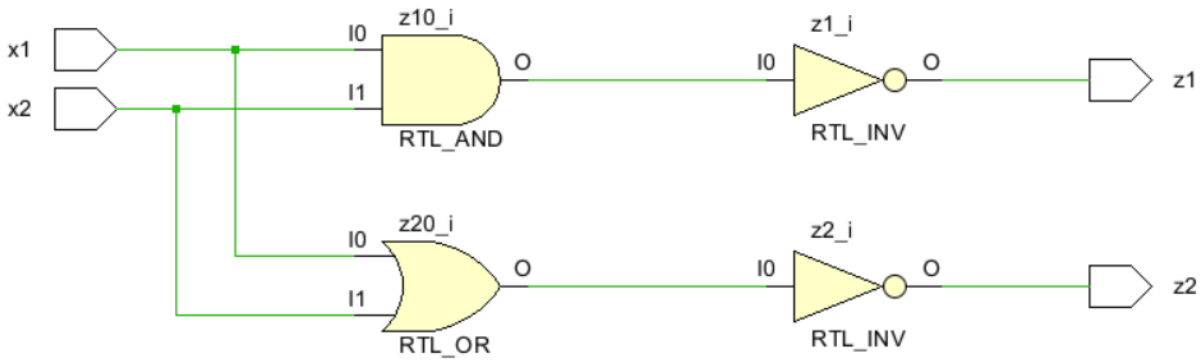
Binary outputs for the test bench



Output waveform

Example 3:

Univaesal gates: nand , nor




```
module univarsal_gates(x1,x2,z1,z2);
   input x1, x2;
   output z1, z2;

   wire x1, x2;
   wire z1, z2;

   assign z1 = ~(x1 & x2);      //nand
   assign z2 = ~(x1 | x2);     //nor

endmodule
```

Verilog module for an univesal gates

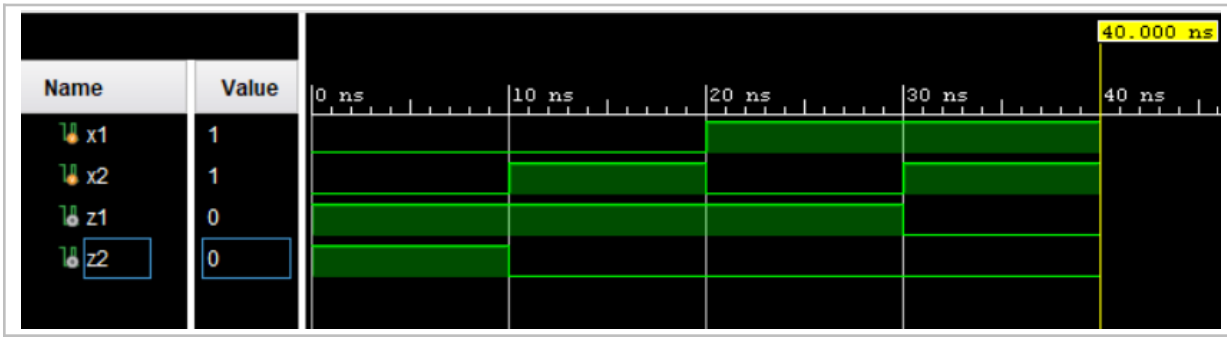
```
module univarsal_gates_tb;

reg x1, x2;
wire z1,z2;

//display variables
initial
$monitor ($time,"ns x1 = %b, x2 = %b, z1 = %b  ,z2=%b", x1, x2,
z1,z2);

//apply input vectors
initial
```

```
begin
#0   x1 = 1'b0;      x2 = 1'b0;
#10  x1 = 1'b0;      x2 = 1'b1;
#10  x1 = 1'b1;      x2 = 1'b0;
#10  x1 = 1'b1;      x2 = 1'b1;
#10 $stop;
end

//instantiate the module into the test bench
univarsal_gates inst1 (
            .x1(x1),
            .x2(x2),
            .z1(z1),
            .z2(z2)
            );
endmodule
```

Test bench for the universal gates

```
# run 1000ns
          0ns x1 = 0, x2 = 0, z1 = 1  ,z2=1
         10ns x1 = 0, x2 = 1, z1 = 1  ,z2=0
         20ns x1 = 1, x2 = 0, z1 = 1  ,z2=0
         30ns x1 = 1, x2 = 1, z1 = 0  ,z2=0
```

Binary outputs for the test bench



waveform

Behavioral modeling:

Highest level of absrtaction provided by HDL.

Behavioural level describes the system by its behaviour.

Two important constructs ‘initial’ and ‘always’

••••••••••••••••

Behavioral Modeling:

1.Describing a module in **behavioral modeling** is an abstraction of the functional operation of the design. It does not describe the implementation of the design at the gate level.

2. The outputs of the module are characterized by their **relationship** to the **inputs**. The behavior of the design is described using procedural constructs.

These constructs are the **initial** statement and the **always** statement.

3.The **initial** statement is executed only **once** during a simulation — beginning at **time 0** — and then **suspends forever**. The **always** statement also begins at **time 0** and executes the statements in the always block **repeatedly** in **a looping manner**. Both statements use only **register** data types.

4.Objects of the reg data types **resemble** the behavior of a hardware storage device because the data **retains** its value until a **new value** is assigned.

5. Behavioral modeling is an algorithmic approach to hardware implementation and represents a higher level of abstraction than previous modeling methods. A Verilog module may contain a mixture of built-in primitives, UDPs, dataflow constructs, and behavioral constructs.

6. A **procedure** is a series of operations taken to design a module. A Verilog module that is designed using behavioral modeling contains no internal structural details, it simply defines the behavior of the hardware in an **abstract**, **algorithmic description**.

7.A **behavior** may consist of a single statement or a block of statements delimited by the keywords **begin** . . . **end**.

```
//set x1
//single statement requires no begin . . . end
initial
   #10   x1 = 1'b1;
```

```
//set x2 and x3
initial
begin
   #10   x2 = 1'b1;
   #10   x3 = 1'b1;
end

//set x4 and x5
initial
begin
   #10   x4 = 1'b1;
   #10   x5 = 1'b1;
end
```

A **module** may contain multiple **initial** and **always** statements.

8.These statements are the basic statements used in behavioral modeling and execute concurrently starting at **time zero** in which the order of execution is not important.

9.All other behavioral statements are contained inside these structured procedure statements.

Initial Statement

1.All statements within an **initial** statement comprise an **initial** block.

2. An initial statement executes only once beginning at **time zero**, then suspends execution.

3.An initial statement provides a method to **initialize** and **monitor** variables before the variables are used in a module; it is also used to **generate waveforms**.

4.For a given **time unit**, all statements within the initial block execute **sequentially**.

5.**Execution** or **assignment** is controlled by the **#** symbol.

Examples of the time symbol are shown below.

(i)At **time zero** (#0), variable **x1** is set to a one-bit (1') binary (b) value of 0.

(ii)**Ten time** units later **x1** and **x2** are set to a value of **1**.

(iii) **Ten time** units later **x1** is set to a value of **0** and **ten time** units later (at 30 time units) x2 is set to a value of 0.

```
#0 x1 = 1'b0;
#10 x1 = 1'b1; x2 = 1'b1;
#10 x1 = 1'b0;
#10 x2 = 1'b0;
```

The syntax for an initial statement is shown below.

initial [optional timing control] procedural statement or block of procedural statements

6.Each initial block executes concurrently at time zero and each block ends execution independently.

7.If there is only one procedural statement, then the statement does not require the keywords **begin . . . end**.
`//set x1`
`//single statement requires no begin . . . end`
`initial`
`  #10  x1 = 1'b1;`
	However, if there are **two** or **more** procedural statements, they are delimited by the keywords **begin . . . end.**

```
//set x2 and x3
initial
begin
   #10   x2 = 1'b1;
   #10   x3 = 1'b1;
end

//set x4 and x5
initial
begin
   #10   x4 = 1'b1;
   #10   x5 = 1'b1;
end
```

Always Statement

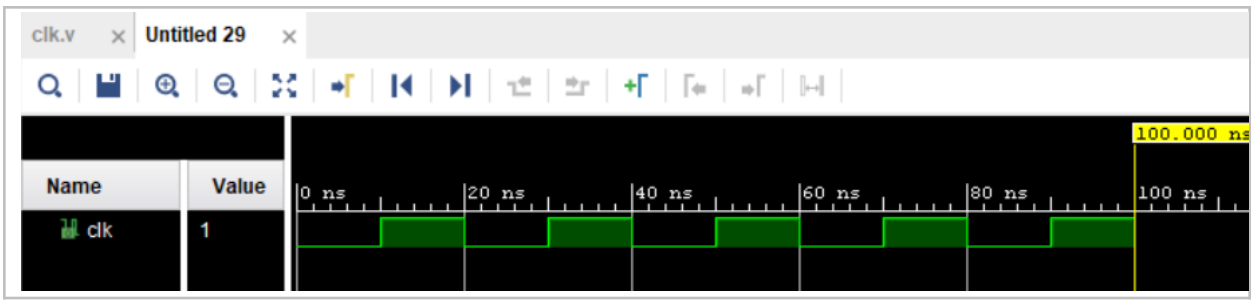
1.The always statement executes the behavioral statements within the always block repeatedly in a looping manner and begins execution at time zero. Execution of the statements continues indefinitely until the simulation is terminated.

A typical application of the **always** statement is shown in Figure to generate a series of **clock** pulses as used in **test bench** constructs. There are two **initial** statements and **one** always statement. The **clock** is first initialized to **zero**, then the **always** statement cycles the clock every **10 time** units for a clock period of **20 time** units. The clock stops cycling after **100 time** units. The system task **$finish** causes the simulator to exit the module and return control to the operating system.

```
//clock generation using initial and always statements
module clk_gen2 (clk);

output clk;
reg clk;

//initialize clock to 0
initial
```

```
clk = 1'b0;

//toggle clock every 10 time units
always
#10 clk = ~clk;

//determine length of simulation
initial
#100 $finish;

endmodule
```

Clock waveform generation



Waveform for the clock generation module

2.An **always** statement is often used with an **event control list** — or **sensitivity list** — to execute a sequential block.

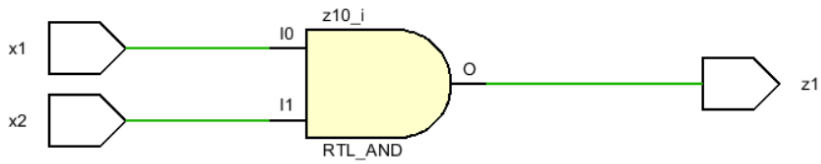
3.When a change occurs to a variable in the sensitivity list, the statement or block of statements in the always block is executed.

3.The keyword **or** is used to indicate multiple events.

4.When **one** or **more** inputs change state, the statement in the always block is executed. The **begin . . . end** keywords are necessary only when there is **more than one** behavioral statement. Target variables used in an **always** statement are declared as type **reg**.

Verilog code for AND gate using Behavioural modeling:

Example 1:




```
//behavioral modelling of and_gate
module  and_gate(x1,x2,z1);

//defining inputs are :x1,x2 and output is :z1
input x1,x2;
output z1;

//declare inputs as type wire and output as type reg
wire x1,x2;
reg z1;

     always @ (x1 or x2) // x1, x2 are  sensitivity list
     begin
     z1=x1&x2;
     end

endmodule
```

Verilog module for and gate

Designing a Test Bench for Simulation:

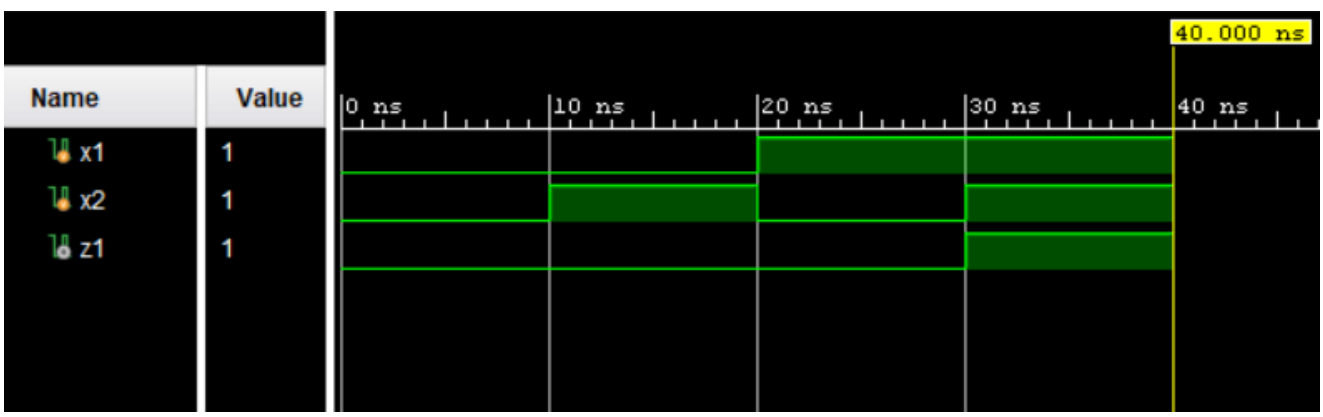
```
//and_gate test bench
module and_gate_tb;
reg x1, x2;
wire z1;

//display variables
initial
$monitor($time,"ns x1 = %b, x2 = %b, z1 = %b", x1, x2, z1);

//apply input vectors
initial
begin
#0  x1 = 1'b0;x2 = 1'b0;
#10 x1 = 1'b0;x2 = 1'b1;
#10 x1 = 1'b1;x2 = 1'b0;
```

```
#10 x1 = 1'b1;x2 = 1'b1;
#10 $stop;
end
//instantiate the module into the test bench
and_gate inst1 (
            .x1(x1),
            .x2(x2),
            .z1(z1)
            );
endmodule
```

Test bench for the and gate

```
# run 1000ns
            0ns x1 = 0, x2 = 0, z1 = 0
            10ns x1 = 0, x2 = 1, z1 = 0
            20ns x1 = 1, x2 = 0, z1 = 0
            30ns x1 = 1, x2 = 1, z1 = 1
```

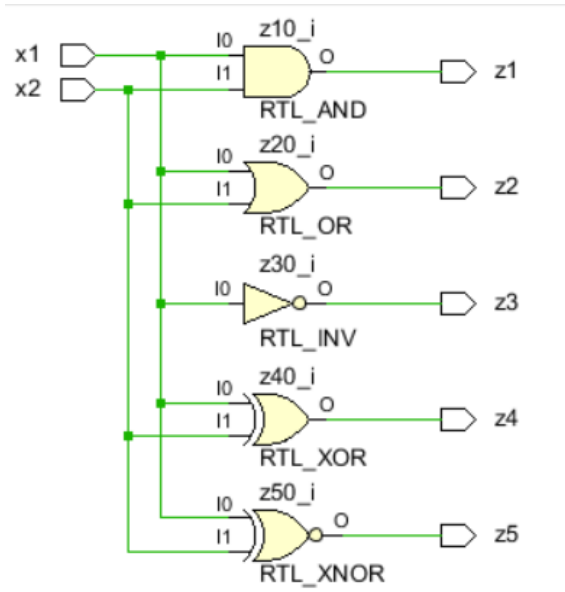
Binary outputs for the test bench



waveform

Example 2:

logic gates:




```
//behavioral modelling of basic_gates
module  basic_gates(x1,x2,z1,z2,z3,z4,z5);

//define inputs x1,x2 and output z1
input x1,x2;
output z1,z2,z3,z4,z5;

//declare inputs as type wire and output as type reg
wire x1,x2;
reg z1,z2,z3,z4,z5;

     always @ (x1 or x2) //x1, x2 are sensitivity lists
     begin
     z1=x1&x2;           //and
     z2=x1|x2;          //or
     z3=~x1;           //not
     z4=x1^x2;        //xor
     z5= x1 ~^ x2;   //xnor
     end

endmodule
```

Verilog module for and gate

Designing a Test Bench for Simulation:

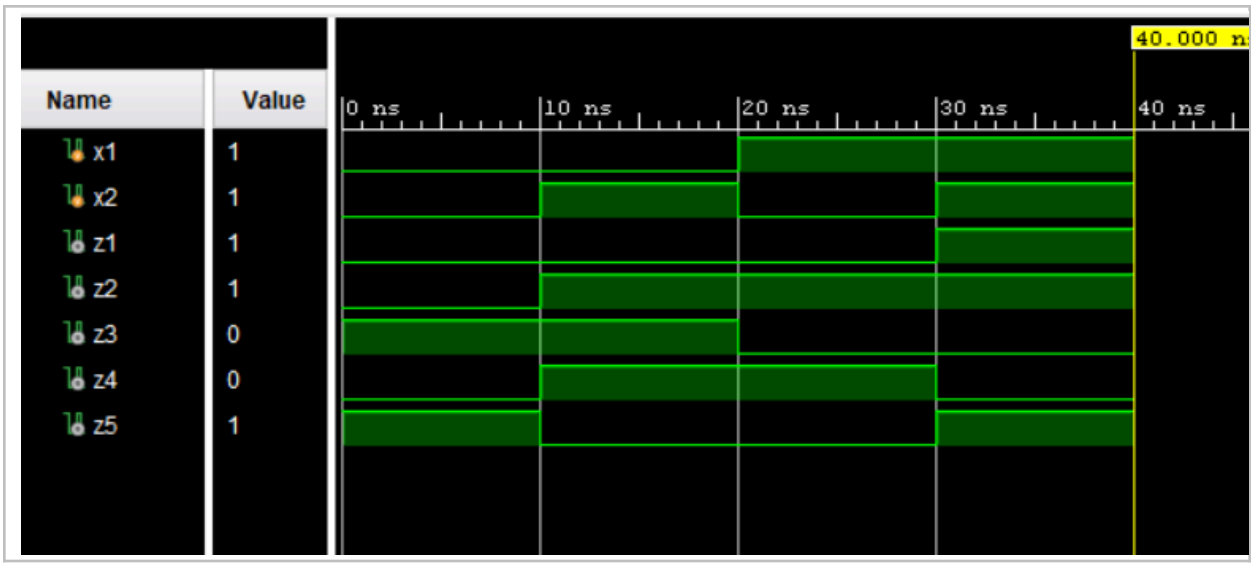
```
//basic_gates test bench
module and_gate_tb;
```

```
reg x1, x2;
wire z1,z2,z3,z4,z5;

//display variables
initial
$monitor($time,"ns x1 = %b, x2 = %b,  &:z1 = %b  ,|:z2=%b,
~:z3=%b ,^:z4=%b ,~^:z5=%b", x1, x2, z1,z2,z3,z4,z5);

//apply input vectors
initial
begin
#0  x1 = 1'b0;x2 = 1'b0;
#10 x1 = 1'b0;x2 = 1'b1;
#10 x1 = 1'b1;x2 = 1'b0;
#10 x1 = 1'b1;x2 = 1'b1;
#10 $stop;
end
//instantiate the module into the test bench
basic_gates inst1 (
            .x1(x1),
            .x2(x2),
            .z1(z1),
            .z2(z2),
            .z3(z3),
            .z4(z4),
            .z5(z5)
            );
endmodule
```

Test bench for the basic gate

```
# run 1000ns
            0ns x1 = 0, x2 = 0,  &:z1 = 0  ,|:z2=0, ~:z3=1 ,^:z4=0 ,~^:z5=1
            10ns x1 = 0, x2 = 1,  &:z1 = 0  ,|:z2=1, ~:z3=1 ,^:z4=1 ,~^:z5=0
            20ns x1 = 1, x2 = 0,  &:z1 = 0  ,|:z2=1, ~:z3=0 ,^:z4=1 ,~^:z5=0
            30ns x1 = 1, x2 = 1,  &:z1 = 1  ,|:z2=1, ~:z3=0 ,^:z4=0 ,~^:z5=1
```

Binary outputs for the test bench



waveform

Gate-level modeling	Data flow level modeling	Behavioural level modeling

3. **Register transfer level** (RTL) is a combination of **dataflow modeling** and **behavioral modeling** and characterizes the flow of data through **logic circuits.**

The following sections describe different techniques used to design logic circuits using dataflow modeling.

These techniques include the

1. Continuous Assignment statement,
2. Reduction operators,
3. The conditional operator,
4. Relational operators,
5. Logical operators,
6. Bitwise operators,
7. Shift operators.

mux

```
/*module  and_gate(x1,x2,z1);
input x1,x2;
```

```
output z1;
wire x1,x2;
reg z1;
always@ (x1 or x2)
    begin
    if (x1 == 1'b1 & x2 == 1'b1)
      begin
      z1 = 1'b0;
      end
      else
      begin
      z1 = 1'b1;
      end
    end
endmodule
*/
```

Intrastatement Delay

An ***intrastatement*** delay is a delay on the **right-hand** side of the statement and indicates that the right-hand side is to be **evaluated**, **wait** the specified number of time units, and then assign the value to the **left-hand** side. This can be used to simulate logic gate delays. Equation below is an example of an intrastatement delay.

y = #5 a & b

The statement evaluates the logical function a **AND** b , waits **five** time units, then assigns the result to **y**. If no **delay** is specified in a procedural assignment, then **zero delay** is the default delay and the assignment occurs instantaneously.

The module of below illustrates the intrastatement delay of five time units for Equation using an always statement. Below shows the test bench in which all combinations of the two inputs are applied to **a** and **b**. The waveforms are shown in Figure that clearly display the intrastatement delay that generates **y** for Equation. When a and b both become asserted, a delay of **five**-time units takes place, then output **y** is asserted.

```
//intrastatement delay
module intrastatement_delay(a, b, y);

//define inputs a,b and output y
input a, b;
output y;

//declare inputs as type wire and output as type reg
```

```
Wire a,b;
reg y;

// event control list – or sensitivity list
always @ (a or b) //a, b are sensitivity lists
   begin
y = #5 (a & b);
   end

endmodule
```

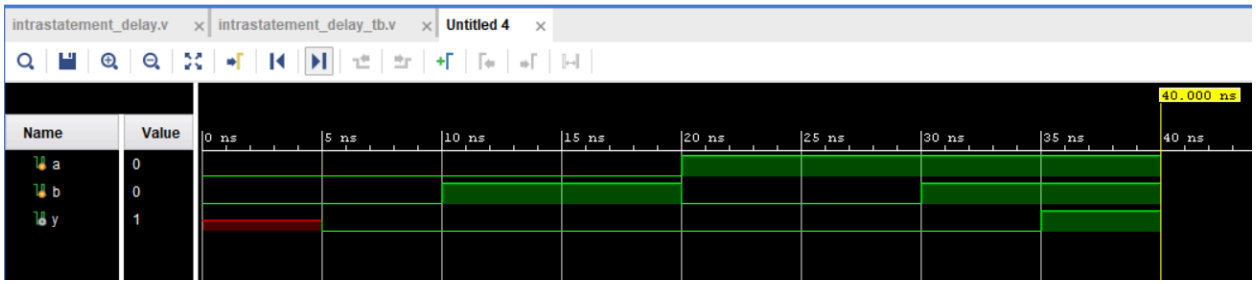
Example of intrastatement delay

```
//test bench of intrastatement delay
module intrastatement_delay_tb;

reg a, b;
wire y;

//display variables
initial
$monitor ("a b = %b, y = %b", {a, b}, y);

//apply input vectors and display variables
initial
begin
#0  a = 1'b0; b = 1'b0;
#10 a = 1'b0; b = 1'b1;
#10 a = 1'b1; b = 1'b0;
#10 a = 1'b1; b = 1'b1;
#10 a = 1'b0; b = 1'b0;
#0 $stop;
end

//instantiate the module into the test bench
intrastatement_delay uut(.a(a),
                         .b(b),
                         .y(y)
                              );
endmodule
```

Test bench

```
# run 1000ns
                    0ns a b = 00, y = x
                    5ns a b = 00, y = 0
                   35ns a b = 11, y = 1
```

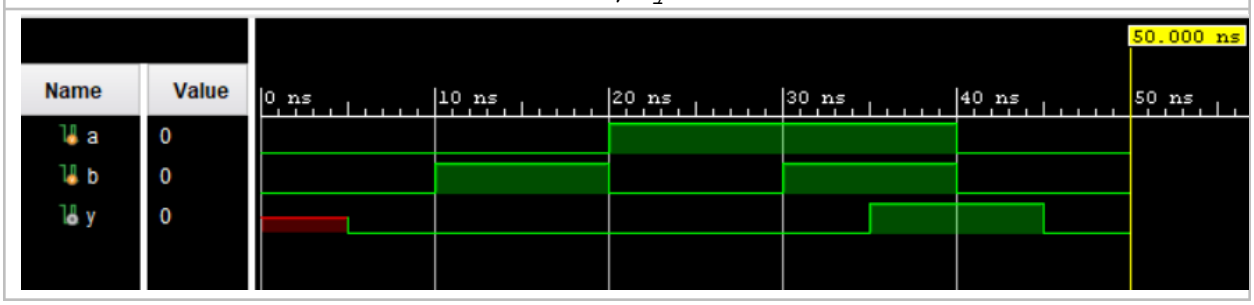
Binary outputs for the test bench




```
//test bench of intrastatement delay
module intrastatement_delay_tb;

reg a, b;
wire y;

//display variables
initial
$monitor ("a b = %b, y = %b", {a, b}, y);

//apply input vectors and display variables
initial
begin
#0  a = 1'b0; b = 1'b0;
#10 a = 1'b0; b = 1'b1;
#10 a = 1'b1; b = 1'b0;
#10 a = 1'b1; b = 1'b1;
#10 a = 1'b0; b = 1'b0;
#10 $stop;
end

//instantiate the module into the test bench
intrastatement_delay uut(.a(a),
                         .b(b),
                         .y(y)
                              );
endmodule
```

```
# run 1000ns
```

```
 0ns a b = 00, y = x
 5ns a b = 00, y = 0
35ns a b = 11, y = 1
45ns a b = 00, y = 0
```



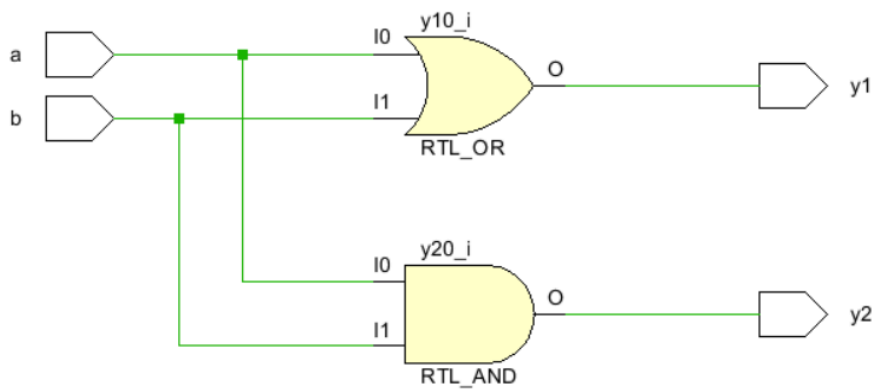
interstatement delay

An interstatement delay is the delay by which a statement's execution is delayed; that is, it is the delay between statements. The code segment of Equation below is an example of an interstatement delay.

y1 = a | b

#5 y2= a & b

When the **first** statement has **completed execution**, a delay of **five-time** units will be taken before the second statement is executed. If **no** delays are specified in a procedural assignment, then there is **zero** delay in the assignment.




```
//behavioral module to illustrate interstatement delay
module interstatement_delay(a, b, y1, y2);
input a, b;
```

```
output y1, y2;
reg y1, y2;
always @ (a or b)
begin
y1 = (a | b);
#5 y2 = (a & b);
end
endmodule
```

Module to illustrate interstatement delay

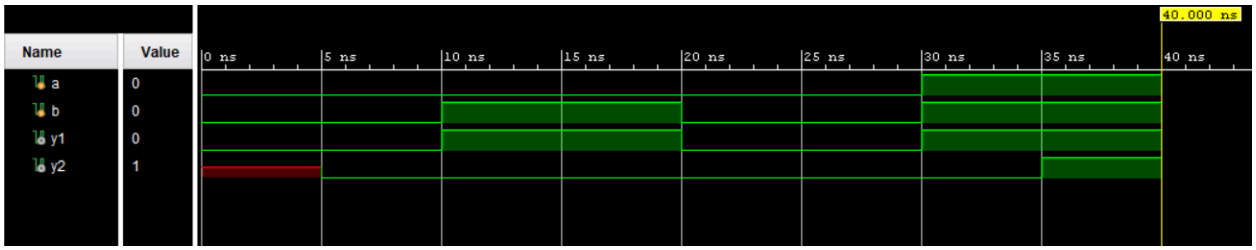
```
//test bench for interstatement delay
module interstatement_delay_tb;

reg a, b;
wire y1, y2;

//display variables
initial
$monitor ("a b = %b, y1 = %b, y2 = %b", {a, b}, y1, y2);
//apply input vectors and display variables
initial
begin
#0  a = 1'b0; b = 1'b0;
#10 a = 1'b0; b = 1'b1;
#10 a = 1'b0; b = 1'b0;
#10 a = 1'b1; b = 1'b1;
#10 a = 1'b0; b = 1'b0;
#0 $stop;
End

//instantiate the module into the test bench
interstatement_delay uut (.a(a),
                          .b(b),
                          .y1(y1),
                          .y2(y2)
                          );
endmodule
```

Test bench

```
# run 1000ns
```

```
a b = 00, y1 = 0, y2 = x
a b = 00, y1 = 0, y2 = 0
a b = 01, y1 = 1, y2 = 0
a b = 00, y1 = 0, y2 = 0
a b = 11, y1 = 1, y2 = 0
a b = 11, y1 = 1, y2 = 1
```

Binary outputs for the test bench



Output waveform

Intrastatement delay indicates that the **statement itself is executed** after the **delay expires,** and it is the most commonly used form of **delay control**. They can be used with blocking and non-blocking assignments. y = #5 a & b	An **interstatement delay** is the delay by which a statement's execution is delayed; that is, it is the delay **between statements**. y1 = a | b #5 y2= a & b

Blocking Assignments

1.A blocking procedural assignment **completes** execution **before** the **next statement** executes. The assignment operator (=) is used for blocking assignments.

2.The **right-hand** expression is evaluated, then the assignment is placed in an internal temporary register called the **event queue** and scheduled for assignment.

3.If **no** time units are specified, then the scheduling takes place immediately.

In the code segment below, an interstatement delay of one time unit is specified for the assignment to z2 . The evaluation of z2 is delayed by the timing control; that is, the expression for z2 will not be evaluated until the expression for z1 has been executed, plus one time unit. The execution of any following statements is blocked until the assignment occurs.

```
initial
begin
z1 = x1 & x2;
#1 z2 = x2 | x3;
end
```

```
module nonblock;
integer a,b;

initial
$monitor($time,"ns %0d %0d",a,b);


always @ (a)
begin
#10 a <= 2'b01;
#10 b <= a;
// all assignments are made here
// b is not (yet) 2'b01
end


endmodule
```

Blocking Assignments and Non-Blocking Assignments:

When working with behavioural modeling in Verilog, there are two types of assigment which is known as **blocking** and **non blocking assigment** and both of them there is a operator, '=' operator for blocking assignment and '<=' operator for non blocking assigment.

At short, **blocking assignment** executes **one by one** sequentially and non-blocking assignemnt executes **parallelly** at the **same time**. Let's see what is the difference between these two in detail , and before going in detail of this, let's us clear some important points about behavioural modeling.

```
module block_block();
 reg a, b, c ;

 initial
 $monitor($time,"ns a=%b b=%b c=%b  ",a, b, c);


 // Blocking assignments
   initial begin
   #0  a = 1'b1;// The simulator assigns 1 to a at time 0
   #10  b = 1'b0;// The simulator assigns 0 to b at time 10
   #10  c =  1'b1;// The simulator assigns 1 to c at time 20
   end
 endmodule
```

```
# run 1000ns
                   0ns a=1 b=x c=x
                  10ns a=1 b=0 c=x
                  20ns a=1 b=0 c=1
```

```
module block_nonblock();
 reg a, b, c ;

 initial
 $monitor($time,"ns a=%b b=%b c=%b  ",a, b, c);


 // Non Blocking assignments
   initial
```

```
   begin
   #0  a <= 1'b1;b <= 1'b0;c <=  1'b1;

   end

 endmodule
```

```
# run 1000ns
                    0ns a=1 b=0 c=1
```

Initial statement:

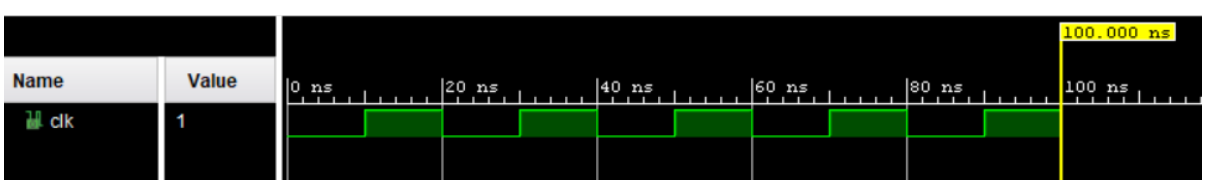
The **initial** statement is executed only **once** during a simulation — beginning at **time 0** — and then **suspends forever**. The **always** statement also begins at **time 0** and executes the statements in the always block **repeatedly** in **a looping manner**. Both statements use only **register** data types.

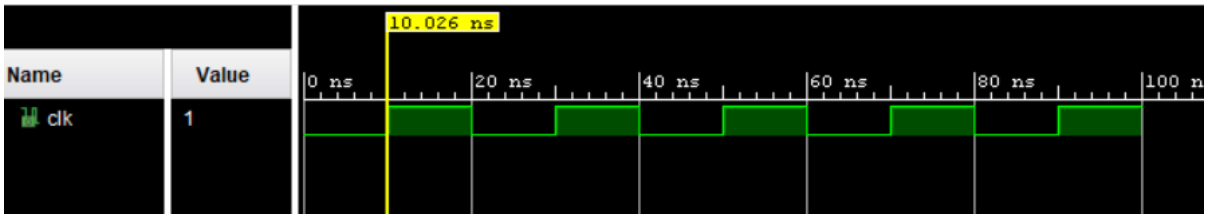
7.If there is only one procedural statement, then the statement does not require the keywords **begin . . . end**.
 	
`// one statement no need begin end keywords`
`initial`
`#0 a=1'b0;`
	However, if there are **two** or **more** procedural statements, they are delimited by the keywords **begin . . . end.**
	
	`initial`
	`begin`
	`#0  a=1'b0'`
	`#10 b=1'b1;`
	`end`
	
	`initial`
	`begin`
	`#10  c=1'b0'`
	`#10 d=1'b1;`
	`#10 e=1'bX;`
	`end`

Always statement

- Is needed for defining memorizing elements (flip-flops, latches)
- Can also be used to define combinational circuits

```
//clock generation
//using initial and always statements

module clk_gen(clk);
output clk;
reg clk;

initial
begin
$monitor($time,"ns clk=%b",clk);
$display("suresh");
end

//initialize clock  at time=0
initial
clk = 1'b0;

//toggle the clock every 10 time units
always
#10 clk = ~clk;

//determine length of simulation (end)
initial
#100 $stop;

endmodule
```

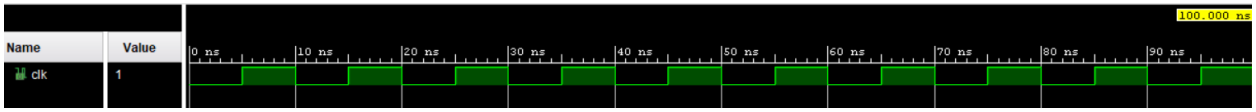





```
# run 1000ns
                    0ns clk=0
                   10ns clk=1
                   20ns clk=0
                   30ns clk=1
                   40ns clk=0
                   50ns clk=1
                   60ns clk=0
                   70ns clk=1
                   80ns clk=0
                   90ns clk=1
```

toggle the clock every 5 time units

```
//clock generation
//using initial and always statements

module clk_gen(clk);
output clk;
reg clk;

initial
$monitor($time,"ns clk=%b",clk);

//initialize clock  at time=0
initial
clk = 1'b0;

//toggle the clock every 10 time units
always
#10 clk = ~clk;

//determine length of simulation (end)
initial
#100 $stop;

endmodule
```

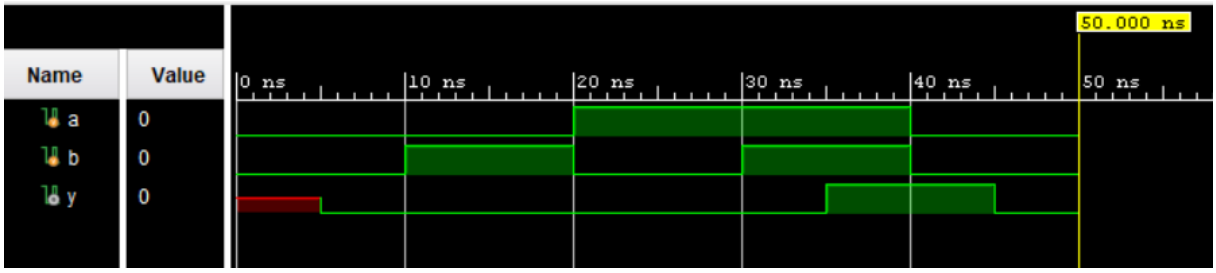


Blocking vs Non-blocking statements

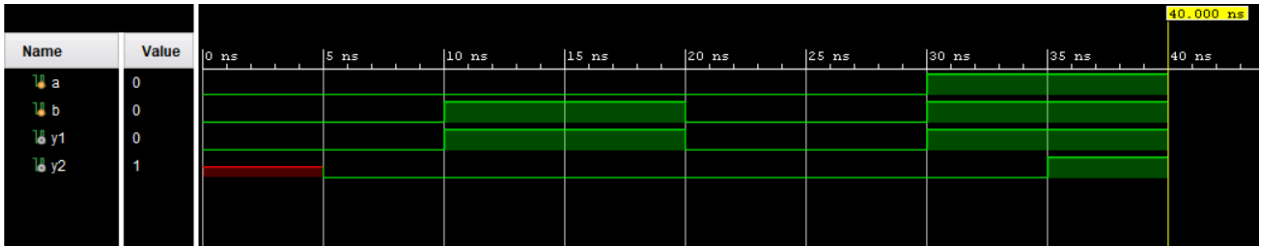
- = assigns the value immediately
- <= assigns the value at the end of the block

Intrastatement delay indicates that the **statement itself is executed** after the **delay expires,** and it is the most commonly used form of **delay control**. They can be used with blocking and non-blocking assignments. y = #5 a & b *See the program*	An **interstatement delay** is the delay by which a statement's execution is delayed; that is, it is the delay **between statements**. y1 = a | b #5 y2= a & b *See the program*
	Intercity express

Intrastatement delay:



interstatement delay:

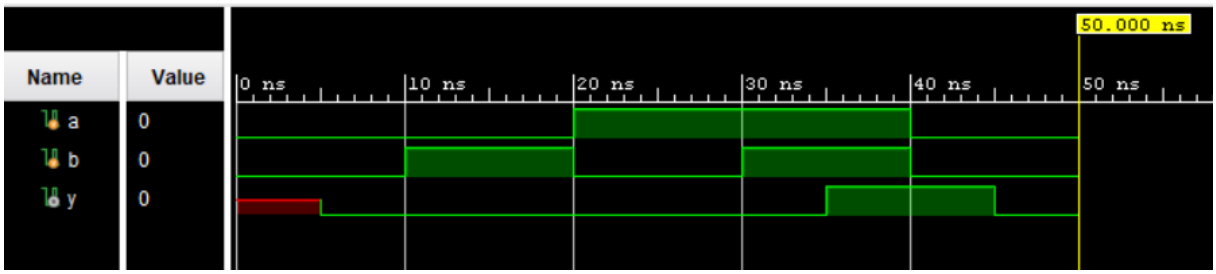



```
//intrastatement delay
module intrastatement_delay(a, b, y);

input a, b;
output y;

Wire a,b;
reg y;
always @ (a or b) //a, b are sensitivity lists
   begin
y = #5 (a & b);
   end

endmodule
```

```
//test bench
module intrastatement_delay_tb;
reg a, b;
wire y;
initial
$monitor ("a b = %b, y = %b", {a, b}, y);

initial
begin
#0  a = 1'b0; b = 1'b0;
#10 a = 1'b0; b = 1'b1;
#10 a = 1'b1; b = 1'b0;
#10 a = 1'b1; b = 1'b1;
#10 a = 1'b0; b = 1'b0;
#0 $stop;
end
```

```
intrastatement_delay uut(.a(a),
                         .b(b),
                         .y(y)
                              );
endmodule
```

```
# run 1000ns
                    0ns a b = 00, y = x
                    5ns a b = 00, y = 0
                   35ns a b = 11, y = 1
```



Interstatement delay:

```
module interstatement_delay(a, b, y1, y2);
input a, b;
output y1, y2;
reg y1, y2;
always @ (a or b)
begin
y1 = (a | b);
#5 y2 = (a & b);
end
endmodule
```

Module to illustrate interstatement delay

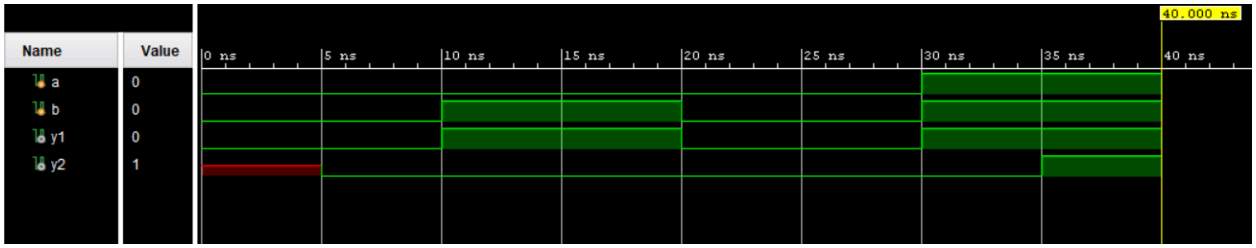
```
//test bench for interstatement delay
module interstatement_delay_tb;
```

```
reg a, b;
wire y1, y2;

//display variables
initial
$monitor ("a b = %b, y1 = %b, y2 = %b", {a, b}, y1, y2);
//apply input vectors and display variables
initial
begin
#0  a = 1'b0; b = 1'b0;
#10 a = 1'b0; b = 1'b1;
#10 a = 1'b0; b = 1'b0;
#10 a = 1'b1; b = 1'b1;
#10 a = 1'b0; b = 1'b0;
#0 $stop;
end

interstatement_delay uut (.a(a),
                          .b(b),
                          .y1(y1),
                          .y2(y2)
                          );
endmodule
```

Test bench

```
# run 1000ns
a b = 00, y1 = 0, y2 = x
a b = 00, y1 = 0, y2 = 0
a b = 01, y1 = 1, y2 = 0
a b = 00, y1 = 0, y2 = 0
a b = 11, y1 = 1, y2 = 0
a b = 11, y1 = 1, y2 = 1
```

Binary outputs for the test bench


Name
Value
a
0
b
0
y1
0
y2
1
0 ns
5 ns
10 ns
15 ns
20 ns
25 ns
30 ns
35 ns
40 ns
40.000 ns